

Explanation

This notebook is gonna be a first of a series of notebooks for the Value Investing Machine Learning project to establish a baseline as a iteration zero for future reference.

Further explanation of the value investing project can be found in the docs -> Domain analysis document

Initial Setup

Below is the install command for the needed libraries to run the notebook

pip install scipy scikit-learn pandas seaborn shap

```
In [1]: import io, math
from scipy.io import arff
from sklearn.metrics import max_error
from sklearn.metrics import mean_squared_error

import sklearn
import pandas as pd
import seaborn
import shap

import numpy as np

import os
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from PIL import Image

import torch
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader, random_split

from torchvision import transforms, models

from sklearn.metrics import classification_report
from sklearn.metrics import multilabel_confusion_matrix
```

```
c:\Users\Ayoub\AppData\Local\Programs\Python\Python313\Lib\site-packages\tqdm\aut
o.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets.
See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

Data provisioning

The dataset consist of 28 columns. Ranging from 7 million to 22 million rows per column, with a total file size of 3.6gb. This immediately shows a processing problem where trying to do anything with the dataset requires alot of compute.

We can try and tackle this. By limiting the columns we actually need from the dataset. For now we gonna just take the colums that have been mentioned in the expert interview or are most closely related.

Also for these columns we dont need to load in all rows. Since the dataset contains a span of 20 years. So ideally we only want to take more recent data. For example the past 3 years.

While doing this we are gonna try and create a new filtered dataset. Consisting of fewer column and only past data rows of 3 years. That filtered dataset we will use further for our model.

Data Dictionary

Identifiers

Ticker: Company trading symbol

Date: Date of the observation (price + ratios)

Price / Market

Adjusted Close (Target Variable): Realistic price including dividends and splits. Used for return calculations.

Volume: Indicates market activity and sentiment around a stock.

Market Cap: Company size. Smaller companies may have more pricing inefficiencies.

Valuation (Core for value investing)

P/E (ttm): Price relative to earnings. Used to assess how expensive a company is.

P/S (ttm): Price relative to revenue. Useful when earnings are less reliable.

P/B: Price relative to assets. Classic value investing metric.

P/FCF (ttm): Price relative to free cash flow.

EV/EBITDA: Measures value based on operational cash flow. Widely used by professionals.

EV/Sales: Alternative valuation when profits are unstable.

Value Signal

Book to Market Value: Inverse of P/B. Indicates how "cheap" a stock is relative to its assets.

Quality (avoid value traps).

Operating Income/EV: Measures profitability relative to company value.

Altman Z Score: Financial health indicator. Helps detect risk of bankruptcy.

Optional

Dividend Yield: Income return from dividends.

Stakeholder mentioned dividend as an important anchor in valuation shows reliability of company.

Data Choice

The dataset contains both quarterly and yearly (TTM) data we will use past data of 3 years.

For this model, TTM data is used because:

It reduces short-term noise

It better reflects long-term earning capacity

It aligns with how the stakeholder evaluates companies with long term vision in mind.

```
In [2]: #data column selection only executed once to select columns
if False:
    df = pd.read_csv("../Dataset/us-derived-shareprices-daily.csv", sep=";")
    df.columns = df.columns.str.strip()

    print(df.columns.tolist())
```

Make sure we select the right columns from the initial dataset.

```
In [3]: # Data filtering step (executed once to create smaller working dataset)
if False:

    # columns we want to use
    cols = [
        "Ticker",
        "Date",
        "Adj. Close",
        "Volume",
        "Market-Cap",
        "Price to Earnings Ratio (ttm)",
        "Price to Sales Ratio (ttm)",
        "Price to Book Value",
        "Price to Free Cash Flow (ttm)",
        "EV/EBITDA",
        "EV/Sales",
        "Book to Market Value",
        "Operating Income/EV",
        "Altman Z Score",
        "Dividend Yield"
    ]

    # Load only selected columns
    df = pd.read_csv(
        "us-derived-shareprices-daily/us-derived-shareprices-daily.csv",
        usecols=cols,
        sep=";"
    )

    # convert Date column
    df["Date"] = pd.to_datetime(df["Date"])

    # filter between 2022 and end of 2025
    df = df[df["Date"].between("2022-01-01", "2025-12-31")]
```

```
# save filtered dataset
df.to_csv("filtered_shares_ratios_3years.csv", index=False)

print("created filtered dataset")

df = pd.read_csv("../filtered_shares_ratios_3years.csv")
df.describe()
```

Out[3]:

	Adj. Close	Volume	Market-Cap	Price to Earnings Ratio (ttm)	Price to Sales Ratio (ttm)	Price
count	5.077223e+06	5.077223e+06	4.846845e+06	3.619868e+06	3.274182e+06	4.0480
mean	2.508054e+04	1.703947e+06	4.219712e+12	-1.730331e+01	1.370486e+02	1.0630
std	1.419652e+06	1.106200e+07	4.197879e+14	2.812195e+03	3.520416e+03	1.9000
min	0.000000e+00	0.000000e+00	0.000000e+00	-1.000000e+05	-1.000000e+05	-1.0000
25%	5.780000e+00	3.290600e+04	1.301741e+08	-3.799602e+00	7.373900e-01	7.8640
50%	1.626000e+01	2.620240e+05	7.850186e+08	6.453625e+00	1.953155e+00	1.6160
75%	4.615000e+01	1.039564e+06	4.506176e+09	2.126395e+01	4.593700e+00	3.6420
max	1.000000e+08	5.497020e+09	5.698460e+16	1.000000e+05	1.000000e+05	1.0000

In this describe even if it is in scientific notation. We can already clearly see in the min and max values huge discrepancies. We have 0 values, as well as huge values seen by the amount of zeros added to some max values.

This suggests that the dataset contains extreme outliers, as well as unusable values for training a model.

Gitignore

Created a filtered dataset reduced from 28 to 15 columns.

Since the datasets are too big to push to github and don't belong in github. Added datasets .csv, .parquet, .xlsx files to gitignore.

In [4]: `df.sample(5)`

Out[4]:

	Ticker	Date	Adj. Close	Volume	Market-Cap	Price to Earnings Ratio (ttm)	Price to Sales Ratio (ttm)	Price to Book Value	P
4520176	TREE	2025-09-02	67.84	248489	9.239917e+08	-16.87329	0.89997	7.71221	8
1751224	FLO	2022-06-16	20.37	1544522	5.211137e+09	23.67397	1.16723	3.57068	16
3052933	MTTR	2024-02-01	2.29	2467782	6.948593e+08	-3.22646	4.36317	1.28736	-8
3940967	RYAN	2024-04-12	49.15	594188	5.640132e+09	-130.02524	2.69476	5.73885	10
2512731	JWN	2024-03-04	19.46	5266640	3.333960e+09	28.11001	0.22732	4.55004	10



Here is a random sample taken from the data which confirms our earlier idea. We can see alot of missing or unusable values as well as very fluctuating values. Since this is the baseline model we won't dive into fixing this.

```
In [5]: (df.isnull().sum() / len(df) * 100).sort_values(ascending=False)
```

```
Out[5]: Dividend Yield          67.768266
Price to Free Cash Flow (ttm)  45.675441
Altman Z Score                43.687228
EV/Sales                     36.955497
EV/EBITDA                    36.731280
Price to Sales Ratio (ttm)    35.512346
Operating Income/EV          29.532719
Price to Earnings Ratio (ttm) 28.703782
Book to Market Value         21.767450
Price to Book Value          20.257826
Market-Cap                   4.537480
Ticker                       0.017194
Volume                       0.000000
Date                         0.000000
Adj. Close                   0.000000
dtype: float64
```

Here we see for each feature the percentage amount of missing values. It's clear that for most values this a way to high number to use in this state for training a model.

Data cleaning and Pre processing notebook

Since this is the baseline model we don't dive to deep into our research how we got to these solutions. However coming to these conclusions took extensive hypothesis and

experimentation to confirm the hypothesis which is covered in the Data cleaning and pre processing notebook.

Preprocessing

Since the dataset is full of outliers, huge differences in values, and missing values or unusable values, as well as text values. This imposes challenging but interesting work for making the dataset suitable for a machine learning model. Since this is still the baseline model, we will keep preprocessing for now as minimally as possible to be able to use the dataset.

```
In [6]: ratio_cols = [
    "Price to Earnings Ratio (ttm)",
    "Price to Sales Ratio (ttm)",
    "Price to Book Value",
    "Price to Free Cash Flow (ttm)",
    "EV/EBITDA",
    "EV/Sales",
    "Book to Market Value",
    "Operating Income/EV",
    "Altman Z Score",
    "Dividend Yield"
]

ticker_quality = (
    df.groupby("Ticker")[ratio_cols]
    .apply(lambda x: x.notna().mean().mean())
    .sort_values(ascending=False)
)

print(ticker_quality)
```

```
Ticker
ZEUS    1.0
ZBH     1.0
ABBV    1.0
AAPL    1.0
YUM     1.0
...
KRNT    0.0
EE      0.0
JSPR    0.0
STG     0.0
ZWS     0.0
Length: 5793, dtype: float64
```

```
In [7]: print("Companies with >= 90% completeness:",
    (ticker_quality >= 0.90).sum())

print("Companies with >= 95% completeness:",
    (ticker_quality >= 0.95).sum())

print("Companies with 100% completeness:",
    (ticker_quality == 1.00).sum())
```

```
print("Total companies:", df["Ticker"].nunique())
```

```
Companies with >= 90% completeness: 2062  
Companies with >= 95% completeness: 844  
Companies with 100% completeness: 681  
Total companies: 5793
```

```
In [8]: complete_tickers = ticker_quality[ticker_quality == 1.0].index.tolist()  
  
print(complete_tickers)  
print("Number of complete companies:", len(complete_tickers))
```

['ZEUS', 'ZBH', 'ABBV', 'AAPL', 'YUM', 'XYL', 'XRX', 'XRAY', 'XPER', 'XOM', 'XLN', 'X', 'XHR', 'WYNN', 'WWW', 'WWE', 'WWD', 'WU', 'ACA', 'ABT', 'WTW', 'WTS', 'WST', 'WSM', 'WRK', 'WNC', 'WMT', 'WMS', 'WMG', 'WMB', 'WM', 'WLFC', 'WIRE', 'WING', 'WINA', 'WHR', 'WHG', 'ACC', 'WHD', 'WGO', 'WEYS', 'WERN', 'WEN', 'WELL', 'WEC', 'WDFC', 'WCN', 'WCC', 'WBA', 'WAB', 'ACN', 'VTRS', 'VST', 'VSEC', 'VRTS', 'VRT', 'VRSK', 'VRS', 'VNT', 'VNO', 'VMI', 'VMC', 'VICI', 'VCTR', 'VATE', 'V', 'UTZ', 'UTMD', 'USLM', 'USG', 'UPS', 'UNP', 'UNH', 'UNF', 'ULH', 'UI', 'UHAL', 'UGI', 'UFP', 'I', 'TXRH', 'TXN', 'TTEK', 'TTEC', 'TTC', 'TT', 'ADP', 'ADI', 'ADES', 'TSN', 'TSCO', 'TRU', 'TRTN', 'TRP', 'TROX', 'TROW', 'TRGP', 'TRCY', 'TR', 'AEE', 'AE', 'TO', 'L', 'TNL', 'TNC', 'TMO', 'TM', 'TKR', 'TJX', 'TILE', 'THO', 'TGT', 'TGLS', 'TER', 'TEL', 'TECH', 'TDG', 'TAP', 'TACO', 'SYY', 'SYK', 'SXT', 'SXC', 'SWM', 'SWKS', 'SWK', 'SWI', 'SWCH', 'SWBI', 'SVC', 'SUP', 'STZ', 'STX', 'AGFS', 'AGCO', 'STRA', 'STOR', 'STLD', 'STE', 'SSTK', 'SSNC', 'SSD', 'SRG', 'SRC', 'SQFTP', 'SPNS', 'SPGI', 'SPG', 'AGX', 'AGTI', 'AGR', 'SOI', 'SNX', 'SNH', 'SNDR', 'SNDK', 'SMP', 'SLCA', 'SLB', 'SJM', 'SJI', 'SIRI', 'SIG', 'SHYF', 'SHW', 'SHOO', 'SHO', 'AIRC', 'AIMEC', 'SEV', 'SEMG', 'SCVL', 'SCU', 'SCS', 'SCL', 'SCI', 'SCHN', 'SCCO', 'SC', 'SBUX', 'AJRD', 'AJG', 'AIT', 'SBAC', 'SAMG', 'SAIC', 'SAFM', 'RYI', 'RUSHA', 'RTX', 'RSG', 'RS', 'RRX', 'RRR', 'RPT', 'RPM', 'ROST', 'ROP', 'ALE', 'ALCO', 'AL', 'ROL', 'ROK', 'ROIC', 'RMD', 'RMAX', 'RLJ', 'RL', 'RGP', 'RFP', 'REYN', 'RESI', 'ALG', 'RELY', 'RELL', 'RCKY', 'RCII', 'QUAD', 'QSR', 'QCOM', 'PXD', 'PWR', 'PWP', 'PVH', 'PTVE', 'PTEN', 'PTE', 'PSX', 'ALLE', 'PSTL', 'PRTH', 'PRIM', 'PRGO', 'POWL', 'POWI', 'POOL', 'PNR', 'PMD', 'PM', 'PLPC', 'PLOW', 'PLD', 'PKI', 'PKG', 'PKE', 'PJT', 'PIPR', 'PH', 'PGRE', 'PFE', 'PEP', 'PEGA', 'PEB', 'PDLI', 'PDCO', 'PDCE', 'PCTI', 'PCH', 'PBI', 'PATK', 'PAHC', 'AMGN', 'AMG', 'AME', 'AMCR', 'PAAS', 'OTTR', 'OTIS', 'OTEX', 'ORCL', 'ORA', 'OMC', 'OLN', 'OHI', 'OC', 'NXST', 'AMP', 'AMOT', 'AMKR', 'NXPI', 'NX', 'NWL', 'NWE', 'NVT', 'NVEC', 'NVDA', 'NUTX', 'NUS', 'NUE', 'NTST', 'NTIC', 'NSC', 'NSA', 'AMTD', 'NRC', 'NOC', 'NMRK', 'NLSN', 'NKE', 'NGVC', 'NEU', 'NEM', 'NEE', 'NDSN', 'NDAQ', 'NBR', 'NATI', 'NATH', 'ANDE', 'MYE', 'MWA', 'MUSA', 'MTRN', 'MTN', 'MTDR', 'MTBC', 'MSM', 'MSI', 'MSFT', 'MSA', 'MRTN', 'MRO', 'MRMD', 'MRLN', 'MRK', 'MPX', 'APAM', 'AOS', 'AON', 'MPC', 'MOS', 'MNRO', 'MNRL', 'MN', 'MMS', 'MMM', 'MLR', 'MLM', 'MLKN', 'MLAB', 'MKSI', 'MKC', 'MIXT', 'APOG', 'APH', 'MIC', 'MGY', 'MGPI', 'MGP', 'MERC', 'MEI', 'MED', 'MDT', 'MDLZ', 'MDC', 'MCY', 'MCO', 'MCK', 'MCHP', 'MCD', 'MC', 'MATX', 'MAS', 'MAPS', 'MANNT', 'MAC', 'MA', 'LW', 'LUMN', 'LSTR', 'LSI_old', 'LRCX', 'ARCO', 'ARCH', 'ARCB', 'ARC', 'LOW', 'LOGI', 'LNN', 'LMT', 'LMNX', 'LMNR', 'LMAT', 'LLY', 'LIN', 'LI', 'I', 'LFUS', 'LEVI', 'ARMK', 'ARKO', 'LEG', 'LECO', 'LEA', 'LDOS_old', 'LDI', 'LCUT', 'LCII', 'LBRT', 'LANC', 'LAMR', 'KW', 'KTB', 'KSS', 'KRG', 'KR', 'KOP', 'AROC', 'KO', 'KNX', 'KMT', 'KMB', 'KLIC', 'KLAC', 'KGC', 'KFY', 'KFRC', 'KEM', 'KDP', 'KBH', 'KBAL', 'KAMN', 'KAI', 'K', 'JW-A', 'JRSH', 'JOUT', 'JOE', 'JNPR', 'JKHY', 'JBL', 'IVZ', 'ITW', 'ITT', 'IPAR', 'IP', 'INTC', 'INT', 'ASTE', 'INGR', 'INFO', 'INDT', 'IMO', 'IFF', 'IEX', 'IDCC', 'ICFI', 'ICE', 'IBP', 'IBM', 'HYAC', 'HY', 'HWM', 'HWKN', 'HVT', 'HURC', 'HTZ', 'HTLD', 'HSY', 'HSII', 'HRL', 'HRB', 'HPQ', 'HPK', 'HPE', 'HP', 'HNNA', 'HMPT', 'HLIO', 'HLI', 'HL', 'HIMX', 'HII', 'ATO', 'HIBB', 'HFC', 'HES', 'HEI', 'HD', 'HCKT', 'HCDI', 'HCC', 'HAYN', 'HAL', 'GW', 'W', 'GVA', 'ATVI', 'GRMN', 'GRC', 'GPT', 'GPS', 'GPRE', 'GPI', 'GPC', 'GNTX', 'GM', 'GLW', 'GILD', 'GHC', 'GGG', 'GES', 'GEN', 'AVB', 'GE', 'GD', 'GCMG', 'GBX', 'FUL', 'FTV', 'FTAI', 'FSS', 'FRG', 'FOX_old', 'FOX', 'FMC', 'FLXS', 'FLS', 'FLR', 'AVGO_old', 'AVD', 'FIX', 'FDS', 'FDP', 'FCFS', 'FBHS', 'FAT', 'FAST', 'FANG', 'EXR', 'EXPD', 'EXP', 'EXE', 'EVTG', 'EVR', 'AVYA', 'AVY', 'EVC', 'EV', 'ETRN', 'ETN', 'ETD', 'ESRT', 'ESCA', 'ES', 'EQIX', 'EPM', 'ENTG', 'ENSG', 'ENS', 'ENR', 'AYI', 'ENB', 'EMR', 'EMN', 'EME', 'EL', 'EFX', 'EE_delisted', 'ECL', 'ECA', 'EBIX', 'EBAY', 'DXPE', 'DVN', 'DUK', 'B', 'AZZ', 'DSKE', 'DS', 'DRI', 'DOX', 'DOW', 'DNUT', 'DKS', 'DHT', 'DHR', 'DHIL', 'DGX', 'DG', 'DFH', 'DE', 'BALL', 'BAH', 'DCI', 'DAN', 'DALN', 'CXW', 'CWEN', 'CW', 'CURO', 'CUB', 'CTXS', 'CTVA', 'CTT', 'CTSH', 'CTS', 'CTAS', 'CSX', 'CSV', 'CSSE', 'BBSI', 'BBBY', 'BAX', 'CSL', 'CSGS', 'CSCO', 'CRS', 'CRI', 'CRD-A', 'CRAI', 'CPLG', 'CPHC', 'COTY', 'COST', 'BBY', 'BBWI', 'CONE', 'COLM', 'COKE', 'CNS', 'CNMD', 'CNLPL', 'CNDT', 'CMI', 'CME', 'CMCSA', 'CMCO', 'CMC', 'CLR', 'BDC', 'BCPC', 'BCO', 'CLAR', 'CL', 'CIVI', 'CIK', 'CHRW', 'CHH', 'CHDN', 'CHD', 'CGNX', 'CFNB', 'CERN', 'CDW', 'CDRE', 'CDK', 'BELFB', 'BDX', 'CCS', 'CCOI', 'CCMP', 'CCJ', 'CCF', 'CBRL', 'CBOE', 'CAT', 'CASY', 'C

```
ASA', 'CAL', 'CAKE', 'CAI', 'CAH', 'CAG', 'CABO', 'BX', 'BW', 'BSY', 'BRMK', 'B
R', 'BNFT', 'BMY', 'BMI', 'BLK', 'BKR', 'BKH', 'BKE', 'BHE', 'BGS', 'BGFV', 'BGC
P', 'BG', 'ZTS', 'AAN', 'AAIC', 'A']
Number of complete companies: 681
```

```
In [9]: print(ticker_quality.sample(20))
```

```
Ticker
PATI    0.000000
SLVM    0.834397
WULF    0.000000
MTLS    0.000000
CCO     0.900000
IGTA    0.275556
FTAI    1.000000
ALLO    0.720538
ADSE    0.000000
TWCB    0.337596
EDN     0.034696
MA      1.000000
NRIM    0.746261
SIF     0.795314
CIG     0.000000
CXDO    0.970090
ID      0.400000
AESE    0.520239
FRG     1.000000
XM      0.900000
dtype: float64
```

Here you can see how much difference there is in the dataset in completeness of the data per company

```
In [10]: good_tickers = ticker_quality[ticker_quality >= 0.90].index

df_good = df[df["Ticker"].isin(good_tickers)]

print("Companies:", df_good["Ticker"].nunique())
print("Rows:", len(df_good))
```

```
Companies: 2062
Rows: 1796191
```

```
In [11]: (df_good.isnull().sum() / len(df_good) * 100).sort_values(ascending=False)
```

```
Out[11]: Dividend Yield      46.957478
         Altman Z Score      5.948477
         Price to Free Cash Flow (ttm)  2.030519
         EV/EBITDA          0.617807
         EV/Sales           0.290893
         Operating Income/EV  0.286439
         Book to Market Value  0.269403
         Price to Book Value   0.238282
         Price to Earnings Ratio (ttm)  0.069703
         Price to Sales Ratio (ttm)    0.062521
         Market-Cap          0.004287
         Volume              0.000000
         Ticker              0.000000
         Date                0.000000
         Adj. Close          0.000000
         dtype: float64
```

```
In [12]: company_df = df_good.groupby("Ticker").last()
         df_good = df_good.drop(columns=["Date", "Ticker", "Dividend Yield"])

         corr = company_df.corr(numeric_only=True)["Adj. Close"]

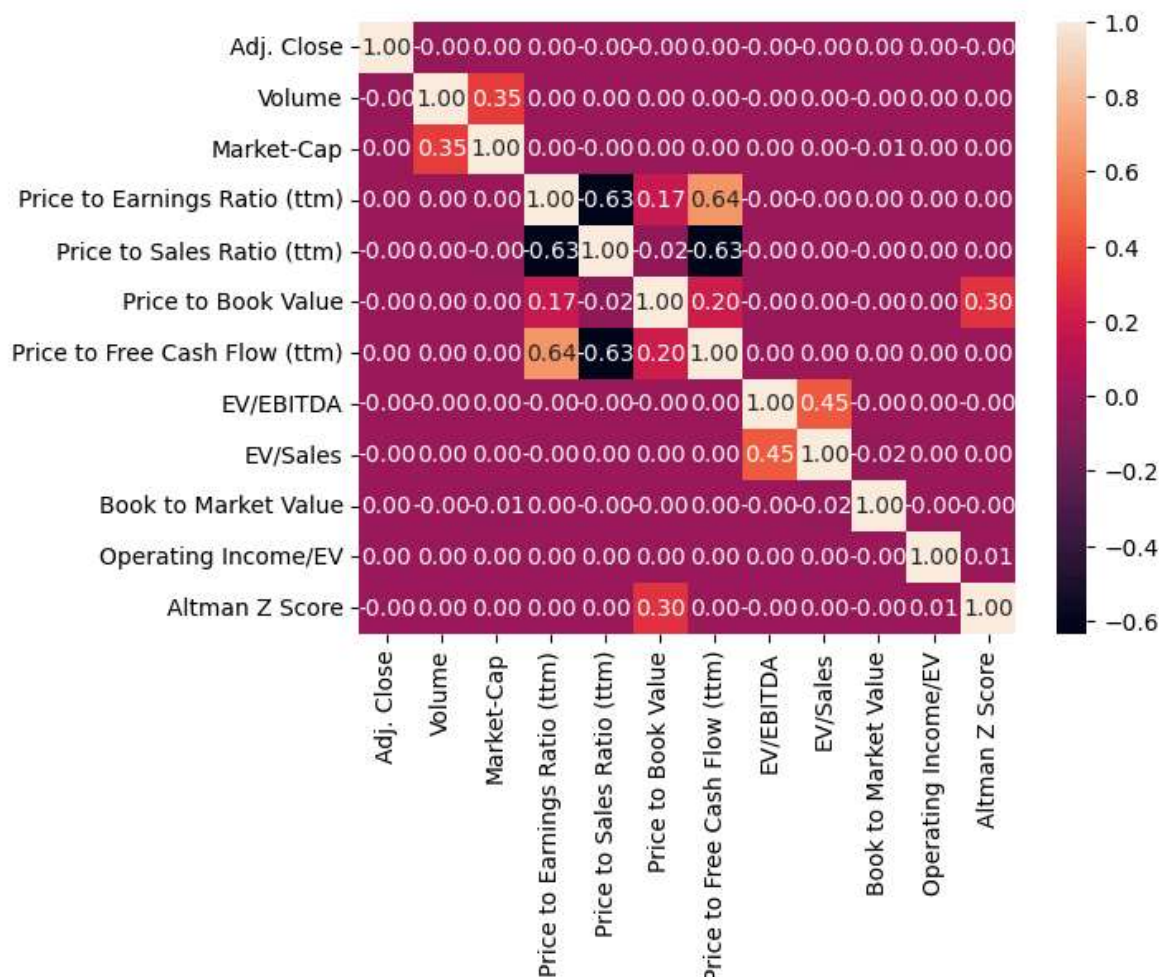
         print(corr.sort_values())
```

```
Volume      -0.006352
Price to Sales Ratio (ttm) -0.001399
EV/Sales     -0.001273
Price to Book Value      -0.000902
Book to Market Value     -0.000397
Operating Income/EV      -0.000391
Price to Earnings Ratio (ttm) -0.000382
Altman Z Score      0.000553
EV/EBITDA      0.000744
Price to Free Cash Flow (ttm)  0.001685
Dividend Yield      0.035076
Market-Cap      0.117756
Adj. Close      1.000000
Name: Adj. Close, dtype: float64
```

```
In [13]: #Check if all columns are the right datatypes
         print(df_good.dtypes)
```

```
Adj. Close      float64
Volume          int64
Market-Cap      float64
Price to Earnings Ratio (ttm) float64
Price to Sales Ratio (ttm)    float64
Price to Book Value      float64
Price to Free Cash Flow (ttm) float64
EV/EBITDA        float64
EV/Sales         float64
Book to Market Value    float64
Operating Income/EV     float64
Altman Z Score      float64
dtype: object
```

```
In [14]: correlations = df_good.corr()
         plot = seaborn.heatmap(correlations, cbar=True, annot=True, fmt=".2f")
```



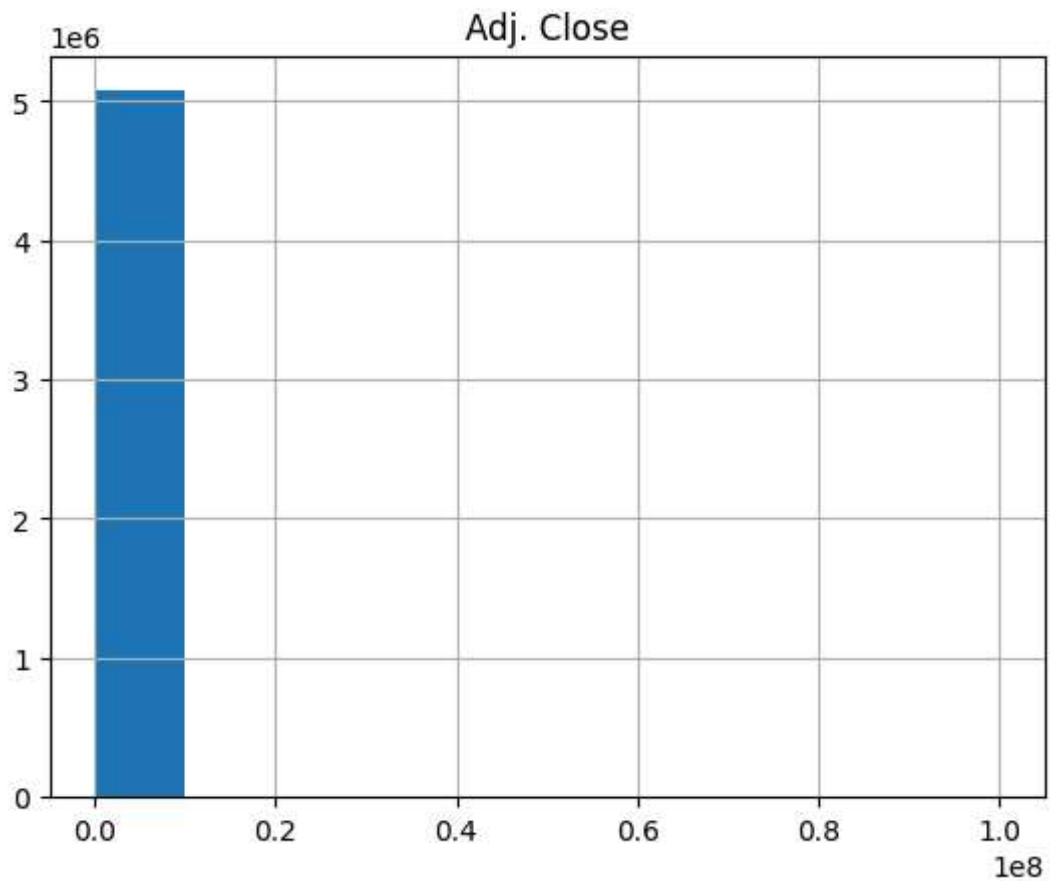
Correlation Matrix with Target variable

In this correlation matrix it shows that the target variable has very weak correlations with the other features. In general all of the features are very weak to none correlation.

However this correlation matrix does not support or domain analysis. This dataset with its features was selected based on the stakeholders input how they analyze businesses and what they look at for stock prices. Since the stakeholder runs a successful investment firm and these practices they've applied for many years.

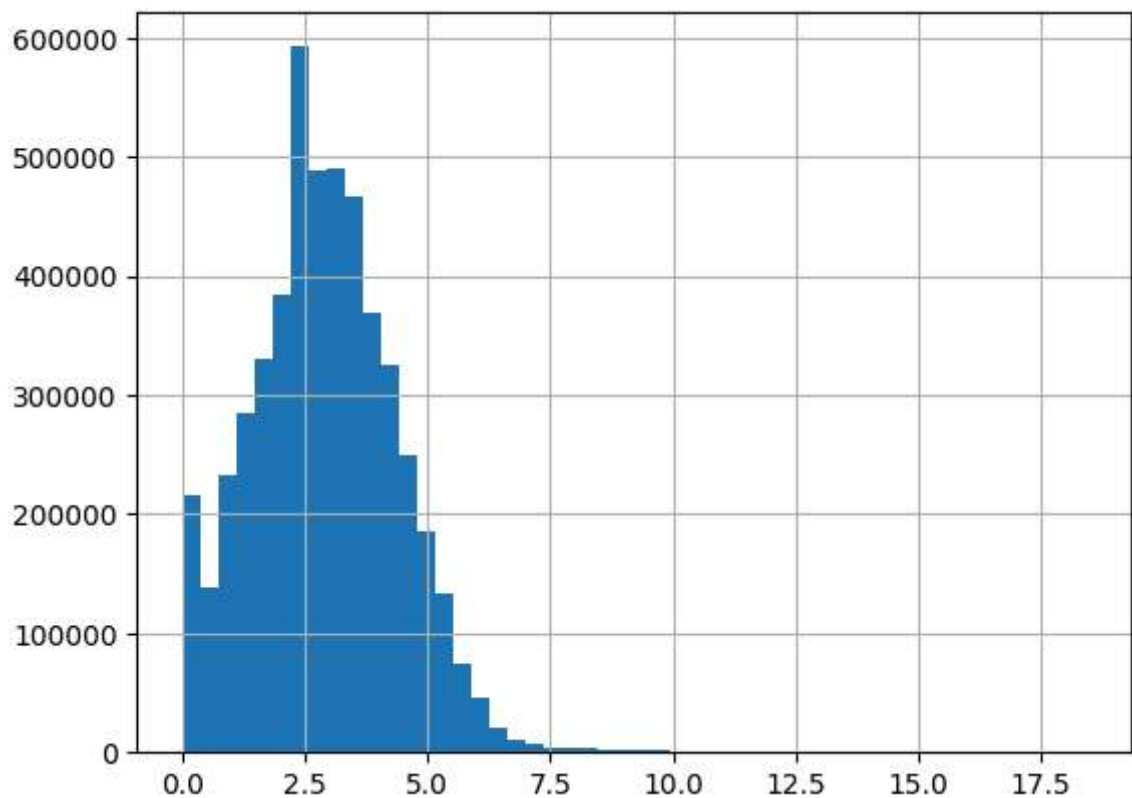
We make the hypothesis that this correlation matrix doesn't accurately represent the features correlations. And there might be an issue in the dataset.

```
In [15]: plot = df.hist(column="Adj. Close")
```

```
In [16]: df["log_adj_close"] = np.log1p(df["Adj. Close"])
df["log_adj_close"].hist(bins=50)
```

Out[16]: <Axes: >



Since the current dataset contains so many missing or unreliable values like -100.000 / 100.000. Or is just heavily influenced by the huge differences between prices. Producing

any visually usable graph without logging is impossible. So we log the data to make the target variable visually understandable.

We can see the data is lightly skewed but mostly normally distributed.

```
In [17]: for col in df_good.select_dtypes(include=["number"]).columns:
          print(f"\n=== {col} ===")
          print(df_good[col].describe(percentiles=[0.01, 0.05, 0.5, 0.95, 0.99, 0.999])

          print("\nTop 10 values:")
          print(df_good[col].sort_values(ascending=False).head(10))

          print("\nBottom 10 values:")
          print(df_good[col].sort_values().head(10))
```

```
=== Adj. Close ===
count    1.796191e+06
mean     1.200725e+04
std      7.959814e+05
min      0.000000e+00
1%       7.400000e-01
5%       2.540000e+00
50%      3.173000e+01
95%      2.809350e+02
99%      6.124410e+02
99.9%    9.424680e+03
max      8.043840e+07
Name: Adj. Close, dtype: float64
```

```
Top 10 values:
4897143    80438400.0
4897203    80136000.0
4897141    79228800.0
4897142    78926400.0
4897140    78624000.0
4897144    78022224.0
4897162    77871024.0
4897155    77716800.0
4897159    77716800.0
4897158    76809600.0
Name: Adj. Close, dtype: float64
```

```
Bottom 10 values:
4247574    0.0
4247573    0.0
4247572    0.0
4247603    0.0
4247602    0.0
4247601    0.0
4247600    0.0
4247610    0.0
4247773    0.0
4247627    0.0
Name: Adj. Close, dtype: float64
```

```
=== Volume ===
count    1.796191e+06
mean     2.445955e+06
std      1.326609e+07
min      0.000000e+00
1%       1.600000e+02
5%       1.090400e+04
50%      5.509920e+05
95%      8.443886e+06
99%      3.042888e+07
99.9%    1.346886e+08
max      1.958425e+09
Name: Volume, dtype: float64
```

```
Top 10 values:
3370563    1958424543
3259213    1543910670
3141214    1326446070
3259030    1178865140
3259275    1156044090
```

```

3259410    1142269060
3259149    1117995410
3370564    1115872737
3370600    1042026148
3259216     998826810
Name: Volume, dtype: int64

```

Bottom 10 values:

```

3513733    0
3513726    0
3513748    0
1050911    0
3513740    0
1050828    0
3513751    0
3513750    0
341994     0
341993     0

```

Name: Volume, dtype: int64

=== Market-Cap ===

```

count    1.796114e+06
mean     2.479151e+10
std      1.443158e+11
min      0.000000e+00
1%       1.152567e+07
5%       6.282568e+07
50%      2.411601e+09
95%      8.758331e+10
99%      3.478954e+11
99.9%    2.607172e+12
max      5.031072e+12

```

Name: Market-Cap, dtype: float64

Top 10 values:

```

3259822    5.031072e+12
3259825    5.027184e+12
3259823    4.930227e+12
3259824    4.920507e+12
3259821    4.885029e+12
3259830    4.836915e+12
3259826    4.828167e+12
3259827    4.743603e+12
3259832    4.709340e+12
3259831    4.693788e+12

```

Name: Market-Cap, dtype: float64

Bottom 10 values:

```

4247662    0.0
4247702    0.0
4247480    0.0
4247479    0.0
4247478    0.0
4247477    0.0
4247701    0.0
4247634    0.0
4247658    0.0
4247657    0.0

```

Name: Market-Cap, dtype: float64

```

=== Price to Earnings Ratio (ttm) ===
count      1.794939e+06
mean       -1.095818e+01
std         2.173083e+03
min         -1.000000e+05
1%          -4.562416e+02
5%          -6.000452e+01
50%         1.322792e+01
95%         8.775440e+01
99%         4.711834e+02
99.9%       9.876856e+03
max         1.000000e+05
Name: Price to Earnings Ratio (ttm), dtype: float64

```

```

Top 10 values:
3123370      99999.99999
3123379      99999.99999
3123380      99999.99999
3123378      99999.99999
3123377      99999.99999
3123376      99999.99999
3123401      99999.99999
3123400      99999.99999
3123399      99999.99999
3123398      99999.99999
Name: Price to Earnings Ratio (ttm), dtype: float64

```

```

Bottom 10 values:
1035616      -99999.99999
1035583      -99999.99999
1035524      -99999.99999
1035494      -99999.99999
3809230      -99999.99999
3809252      -99999.99999
3809248      -99999.99999
3809119      -99999.99999
3809088      -99999.99999
1035510      -99999.99999
Name: Price to Earnings Ratio (ttm), dtype: float64

```

```

=== Price to Sales Ratio (ttm) ===
count      1.795068e+06
mean        2.473122e+01
std         8.502281e+02
min         -4.142105e+03
1%          3.671000e-02
5%          1.866900e-01
50%         1.878315e+00
95%         1.219603e+01
99%         4.271511e+01
99.9%       2.955692e+03
max         1.000000e+05
Name: Price to Sales Ratio (ttm), dtype: float64

```

```

Top 10 values:
1035503      99999.99999
1035486      99999.99999
1035487      99999.99999
1035502      99999.99999
1035501      99999.99999

```

```

1035504    99999.99999
1035505    99999.99999
1035510    99999.99999
1035506    99999.99999
1035507    99999.99999

```

Name: Price to Sales Ratio (ttm), dtype: float64

Bottom 10 values:

```

3618136    -4142.10515
3618134    -3979.42429
3618135    -3960.65342
3618143    -3948.13950
3618132    -3929.36863
3618133    -3916.85472
3618140    -3898.08385
3618144    -3873.05603
3618146    -3848.02820
3618137    -3848.02820

```

Name: Price to Sales Ratio (ttm), dtype: float64

=== Price to Book Value ===

```

count      1.791911e+06
mean        8.781438e+00
std         1.253199e+03
min         -1.000000e+05
1%          -4.072795e+01
5%          -1.638200e+00
50%          2.115760e+00
95%          1.799916e+01
99%          1.154061e+02
99.9%       3.400502e+03
max         1.000000e+05

```

Name: Price to Book Value, dtype: float64

Top 10 values:

```

1035502    99999.99999
1035499    99999.99999
1035498    99999.99999
1035501    99999.99999
1035500    99999.99999
1035515    99999.99999
478497     99999.99999
478496     99999.99999
478495     99999.99999
1035486     99999.99999

```

Name: Price to Book Value, dtype: float64

Bottom 10 values:

```

3809034    -99999.99999
3809035    -99999.99999
3809036    -99999.99999
3809037    -99999.99999
3809120    -99999.99999
3809104    -99999.99999
3809014    -99999.99999
3809013    -99999.99999
3809109    -99999.99999
3809140    -99999.99999

```

Name: Price to Book Value, dtype: float64

=== Price to Free Cash Flow (ttm) ===

```
count    1.759719e+06
mean     -1.983423e+01
std       2.237992e+03
min       -1.000000e+05
1%        -3.954663e+02
5%        -5.402822e+01
50%       7.772720e+00
95%       7.071124e+01
99%       4.568971e+02
99.9%     7.905892e+03
max       1.000000e+05
```

Name: Price to Free Cash Flow (ttm), dtype: float64

Top 10 values:

```
1866818    99999.99999
1866806    99999.99999
1866807    99999.99999
1866808    99999.99999
1866819    99999.99999
1866820    99999.99999
1866821    99999.99999
1866822    99999.99999
1866823    99999.99999
1866824    99999.99999
```

Name: Price to Free Cash Flow (ttm), dtype: float64

Bottom 10 values:

```
1035506    -99999.99999
3809208    -99999.99999
1035550    -99999.99999
1035551    -99999.99999
1035558    -99999.99999
1035559    -99999.99999
1035562    -99999.99999
1035561    -99999.99999
1035540    -99999.99999
1035541    -99999.99999
```

Name: Price to Free Cash Flow (ttm), dtype: float64

=== EV/EBITDA ===

```
count    1.785094e+06
mean     1.319890e+01
std       9.914837e+02
min       -1.000000e+05
1%        -1.867758e+02
5%        -2.748448e+01
50%       1.033638e+01
95%       5.060655e+01
99%       2.089176e+02
99.9%     1.925207e+03
max       1.000000e+05
```

Name: EV/EBITDA, dtype: float64

Top 10 values:

```
1089376    99999.99999
1089375    99999.99999
1089389    99999.99999
1089388    99999.99999
1089387    99999.99999
```

```

1089386    99999.99999
1089385    99999.99999
1089384    99999.99999
1089383    99999.99999
1089382    99999.99999
Name: EV/EBITDA, dtype: float64

```

```

Bottom 10 values:
1366161    -99999.99999
1366173    -99999.99999
1366171    -99999.99999
1366172    -99999.99999
1366160    -99999.99999
1366152    -99999.99999
1366151    -99999.99999
1366150    -99999.99999
1366154    -99999.99999
1366116    -99999.99999
Name: EV/EBITDA, dtype: float64

```

```

=== EV/Sales ===
count      1.790966e+06
mean        6.446496e+00
std         8.082758e+02
min        -1.000000e+05
1%         -2.169800e-01
5%          2.742800e-01
50%         2.206215e+00
95%         1.303504e+01
99%         2.827954e+01
99.9%       3.127665e+02
max         1.000000e+05
Name: EV/Sales, dtype: float64

```

```

Top 10 values:
1089389    99999.99999
1089375    99999.99999
1089376    99999.99999
1089423    99999.99999
1089422    99999.99999
1089421    99999.99999
1089388    99999.99999
1089387    99999.99999
1089386    99999.99999
1089385    99999.99999
Name: EV/Sales, dtype: float64

```

```

Bottom 10 values:
2531543    -99999.99999
2531574    -99999.99999
2531578    -99999.99999
2531545    -99999.99999
2531546    -99999.99999
2531547    -99999.99999
2531529    -99999.99999
2531552    -99999.99999
2531535    -99999.99999
2531544    -99999.99999
Name: EV/Sales, dtype: float64

```

=== Book to Market Value ===

```
count    1.791352e+06
mean     1.376852e+00
std      1.411852e+02
min      -1.666667e+04
1%       -1.029589e+00
5%       -3.522000e-02
50%      3.988400e-01
95%      1.867660e+00
99%      7.183390e+00
99.9%    7.246377e+02
max      7.692308e+03
```

Name: Book to Market Value, dtype: float64

Top 10 values:

```
4897866    7692.30769
4897868    7692.30769
4897861    7692.30769
4897863    7142.85714
4897871    7142.85714
4897867    7142.85714
4897864    7142.85714
4897865    7142.85714
4897870    7142.85714
4897862    6666.66667
```

Name: Book to Market Value, dtype: float64

Bottom 10 values:

```
2850466   -16666.66667
2850467   -16666.66667
2850468   -16666.66667
2850469   -16666.66667
2850465   -16666.66667
2850424   -16666.66667
2850449   -16666.66667
2850432   -16666.66667
2850435   -16666.66667
2850450   -16666.66667
```

Name: Book to Market Value, dtype: float64

=== Operating Income/EV ===

```
count    1.791046e+06
mean     -2.947662e-01
std      3.645276e+01
min      -1.164250e+04
1%       -1.537027e+00
5%       -2.364500e-01
50%      4.413000e-02
95%      1.969100e-01
99%      7.878555e-01
99.9%    1.205159e+01
max      7.242690e+03
```

Name: Operating Income/EV, dtype: float64

Top 10 values:

```
2217460    7242.68974
1207596    7170.05208
4493259    5106.50711
4176274    4843.71608
3168362    3969.08215
```



```

3168361    3969.08215
3168356    3969.08215
4244869    2002.21108
4175799    1927.00422
1070596    1756.61255
Name: Operating Income/EV, dtype: float64

```

```

Bottom 10 values:
1822844    -11642.50191
900057     -5464.30564
900055     -5464.30564
900056     -5090.36251
4244867    -5066.71585
4244880    -5066.71585
900058     -4764.32191
900053     -4764.32191
900054     -4764.32191
899826     -4581.07719
Name: Operating Income/EV, dtype: float64

```

```

=== Altman Z Score ===
count      1.689345e+06
mean        2.192986e+00
std          4.372187e+02
min         -6.610911e+03
1%          -9.501250e+00
5%          -1.162760e+00
50%          2.639200e+00
95%          1.282837e+01
99%          3.096765e+01
99.9%       1.167107e+02
max          1.000000e+05
Name: Altman Z Score, dtype: float64

```

```

Top 10 values:
478510     99999.99999
478511     99999.99999
478512     99999.99999
478507     99999.99999
478506     99999.99999
478505     99999.99999
478504     99999.99999
478508     99999.99999
478514     99999.99999
478496     99999.99999
Name: Altman Z Score, dtype: float64

```

```

Bottom 10 values:
4716098    -6610.91087
4716099    -6610.90298
4716097    -6610.90035
4716096    -6610.89071
4716100    -6610.88545
4716101    -6610.88019
4716102    -6610.87756
4716103    -6610.86705
4716091    -6610.85302
4716095    -6610.84514
Name: Altman Z Score, dtype: float64

```

Shows that the entire dataset each feature contains extreme outlier and placeholder values for NaN values making the dataset unusable in it's current state.

Workaround

You can also try and work the other way around instead of arbitraly cleaning up ur dataset. Since this is the baseline model I'll keep it short but in the, preprocessing notebook you can see my attempts at cleaning the dataset, and how I pivoted to looking for right data instead of wrong data.

You can just look in the dataset for companies that contain all financial ratios and data. and then just use those companies for the modeling.

```
In [18]: df_trim = df_good.copy()

df_trim.drop(
    columns=["log_adj_close"],
    errors="ignore"
).copy()

numeric_cols = df_trim.select_dtypes(include=["number"]).columns

for col in numeric_cols:

    lower = df_trim[col].quantile(0.01)
    upper = df_trim[col].quantile(0.99)

    df_trim = df_trim[
        (df_trim[col] >= lower) &
        (df_trim[col] <= upper)
    ]

print("Original shape:", df_good.shape)
print("Trimmed shape:", df_trim.shape)
```

Original shape: (1796191, 12)

Trimmed shape: (1296689, 12)

Here we arbitrarily eliminate bottom and top 0,5% of the dataset to remove extreme values

```
In [19]: corr_original = (
    df_good
    .corr(numeric_only=True)["Adj. Close"]
    .sort_values()
)

corr_trimmed = (
    df_trim
    .corr(numeric_only=True)["Adj. Close"]
    .sort_values()
)

print("ORIGINAL")
print(corr_original)
```

```
print("\nTRIMMED")
print(corr_trimmed)
```

```
ORIGINAL
Volume                -0.002765
Price to Sales Ratio (ttm) -0.000432
EV/EBITDA             -0.000189
Altman Z Score        -0.000116
EV/Sales              -0.000114
Price to Book Value   -0.000102
Market-Cap            0.000029
Price to Earnings Ratio (ttm) 0.000057
Price to Free Cash Flow (ttm) 0.000129
Book to Market Value  0.000772
Operating Income/EV   0.001447
Adj. Close            1.000000
Name: Adj. Close, dtype: float64
```

```
TRIMMED
Book to Market Value   -0.312977
Volume                -0.063993
Operating Income/EV    0.102269
Price to Free Cash Flow (ttm) 0.135335
Price to Earnings Ratio (ttm) 0.147125
EV/EBITDA             0.154956
Price to Book Value    0.223199
Altman Z Score         0.273470
EV/Sales              0.273738
Price to Sales Ratio (ttm) 0.295118
Market-Cap            0.432477
Adj. Close            1.000000
Name: Adj. Close, dtype: float64
```

```
In [20]: print(
        corr_original.abs().sort_values(ascending=False)
    )
```

```
Adj. Close            1.000000
Volume                0.002765
Operating Income/EV   0.001447
Book to Market Value  0.000772
Price to Sales Ratio (ttm) 0.000432
EV/EBITDA            0.000189
Price to Free Cash Flow (ttm) 0.000129
Altman Z Score        0.000116
EV/Sales              0.000114
Price to Book Value   0.000102
Price to Earnings Ratio (ttm) 0.000057
Market-Cap            0.000029
Name: Adj. Close, dtype: float64
```

```
In [21]: print(
        corr_trimmed.abs().sort_values(ascending=False)
    )
```

Adj. Close	1.000000
Market-Cap	0.432477
Book to Market Value	0.312977
Price to Sales Ratio (ttm)	0.295118
EV/Sales	0.273738
Altman Z Score	0.273470
Price to Book Value	0.223199
EV/EBITDA	0.154956
Price to Earnings Ratio (ttm)	0.147125
Price to Free Cash Flow (ttm)	0.135335
Operating Income/EV	0.102269
Volume	0.063993

Name: Adj. Close, dtype: float64

```
In [22]: for col in df_trim.columns:
          print(f"\n{'='*60}")
          print(col)
          print('='*60)

          print(
              df_trim[col].describe(
                  percentiles=[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]
              )
          )
```

=====

Adj. Close

=====

count	1.296689e+06
mean	6.939468e+01
std	9.050800e+01
min	7.400000e-01
1%	1.650000e+00
5%	3.750000e+00
25%	1.349000e+01
50%	3.478000e+01
75%	8.702000e+01
95%	2.499100e+02
99%	4.656112e+02
max	6.124400e+02

Name: Adj. Close, dtype: float64

=====

Volume

=====

count	1.296689e+06
mean	1.666586e+06
std	3.177014e+06
min	3.780000e+02
1%	3.433000e+03
5%	2.412800e+04
25%	1.963130e+05
50%	5.937740e+05
75%	1.612924e+06
95%	7.212212e+06
99%	1.731061e+07
max	3.060176e+07

Name: Volume, dtype: float64

=====

Market-Cap

=====

count	1.296689e+06
mean	1.362685e+10
std	3.091837e+10
min	1.762183e+07
1%	3.931256e+07
5%	1.129754e+08
25%	7.513133e+08
50%	2.745254e+09
75%	1.031432e+10
95%	6.853149e+10
99%	1.752857e+11
max	2.481752e+11

Name: Market-Cap, dtype: float64

=====

Price to Earnings Ratio (ttm)

=====

count	1.296689e+06
mean	1.462635e+01
std	5.311656e+01
min	-4.641648e+02
1%	-1.661900e+02
5%	-4.224908e+01

```

25%      -1.294970e+00
50%       1.504123e+01
75%       2.787351e+01
95%       7.375660e+01
99%       1.964540e+02
max       4.691106e+02

```

Name: Price to Earnings Ratio (ttm), dtype: float64

```

=====
Price to Sales Ratio (ttm)
=====

```

```

count      1.296689e+06
mean       2.657367e+00
std        2.810448e+00
min        6.185000e-02
1%         1.132400e-01
5%         2.466200e-01
25%        7.587900e-01
50%        1.683410e+00
75%        3.463090e+00
95%        8.736860e+00
99%        1.339222e+01
max        2.298886e+01

```

Name: Price to Sales Ratio (ttm), dtype: float64

```

=====
Price to Book Value
=====

```

```

count      1.296689e+06
mean       3.496725e+00
std        6.440195e+00
min       -3.759563e+01
1%       -1.567188e+01
5%        3.450700e-01
25%       1.263180e+00
50%       2.313270e+00
75%       4.459600e+00
95%       1.295927e+01
99%       2.874507e+01
max       5.573672e+01

```

Name: Price to Book Value, dtype: float64

```

=====
Price to Free Cash Flow (ttm)
=====

```

```

count      1.296689e+06
mean       9.230211e+00
std        3.718932e+01
min       -2.632669e+02
1%       -1.137634e+02
5%       -3.776612e+01
25%       1.809430e+00
50%       8.905800e+00
75%       1.823255e+01
95%       5.331931e+01
99%       1.472554e+02
max       2.887827e+02

```

Name: Price to Free Cash Flow (ttm), dtype: float64

```

=====

```

EV/EBITDA

```
=====
count      1.296689e+06
mean       1.024265e+01
std        2.236368e+01
min        -1.440003e+02
1%         -7.856247e+01
5%         -2.106067e+01
25%        5.878020e+00
50%        1.048524e+01
75%        1.677987e+01
95%        3.790547e+01
99%        8.251450e+01
max        1.402199e+02
Name: EV/EBITDA, dtype: float64
```

EV/Sales

```
=====
count      1.296689e+06
mean       3.042060e+00
std        3.006644e+00
min        -1.239000e-02
1%         1.876800e-01
5%         3.751840e-01
25%        9.890500e-01
50%        2.016820e+00
75%        3.938320e+00
95%        9.706034e+00
99%        1.434651e+01
max        1.696576e+01
Name: EV/Sales, dtype: float64
```

Book to Market Value

```
=====
count      1.296689e+06
mean       5.230143e-01
std        5.185658e-01
min        -6.625900e-01
1%         -2.925300e-01
5%         2.729000e-02
25%        1.922900e-01
50%        3.910500e-01
75%        6.990200e-01
95%        1.506180e+00
99%        2.572280e+00
max        3.720100e+00
Name: Book to Market Value, dtype: float64
```

Operating Income/EV

```
=====
count      1.296689e+06
mean       3.817188e-02
std        1.085882e-01
min        -7.448800e-01
1%         -4.212400e-01
5%         -1.389900e-01
25%        1.892000e-02
```

```

50%      4.823000e-02
75%      8.134000e-02
95%      1.731700e-01
99%      2.882200e-01
max       4.235500e-01
Name: Operating Income/EV, dtype: float64

```

```
=====
Altman Z Score
=====
```

```

count      1.296689e+06
mean       3.598000e+00
std        3.588722e+00
min       -5.068860e+00
1%        -2.203336e+00
5%         2.583400e-02
25%        1.595800e+00
50%        2.761320e+00
75%        4.477850e+00
95%        1.057991e+01
99%        1.873671e+01
max        2.562292e+01
Name: Altman Z Score, dtype: float64

```

```
In [23]: import matplotlib.pyplot as plt
```

```

for col in df_good.columns:

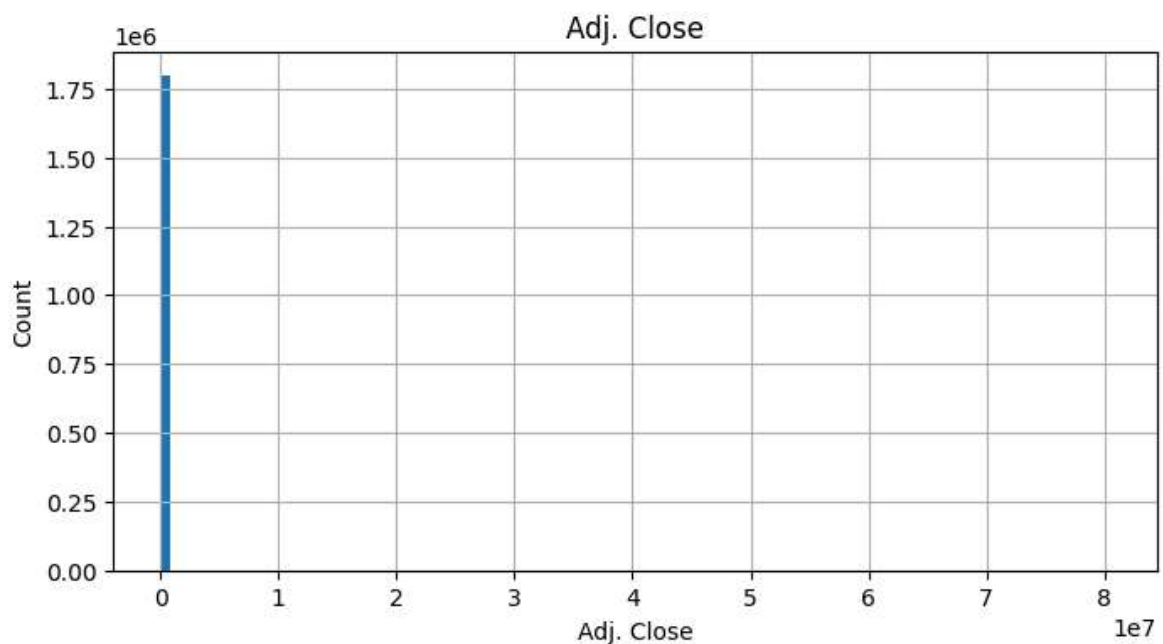
    plt.figure(figsize=(8, 4))

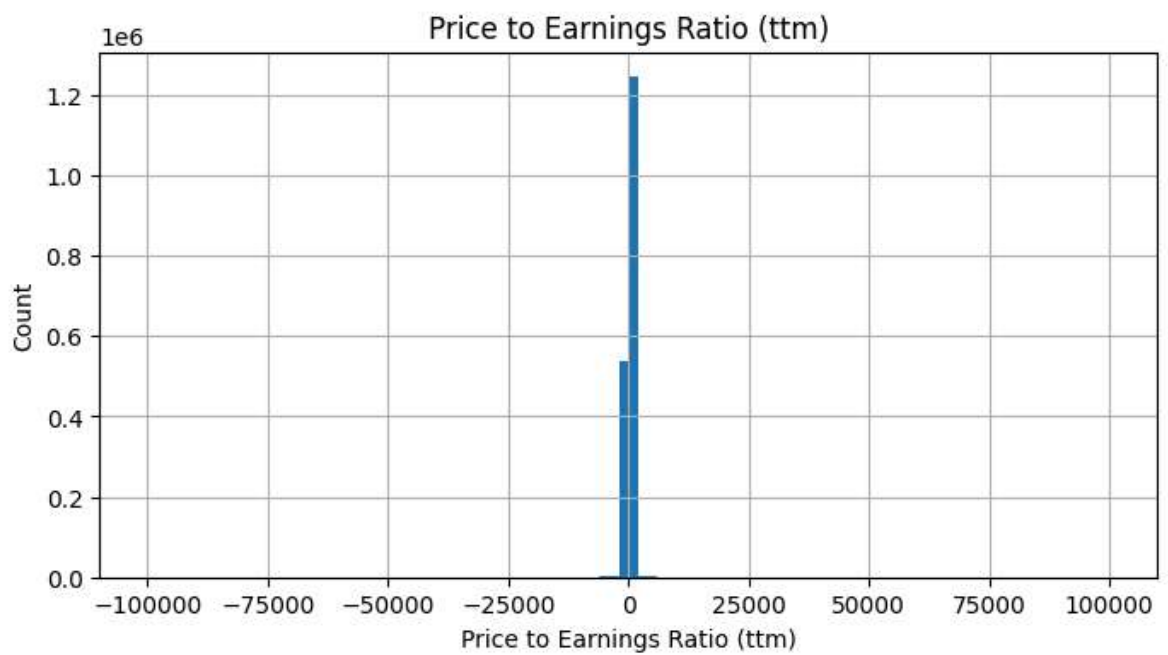
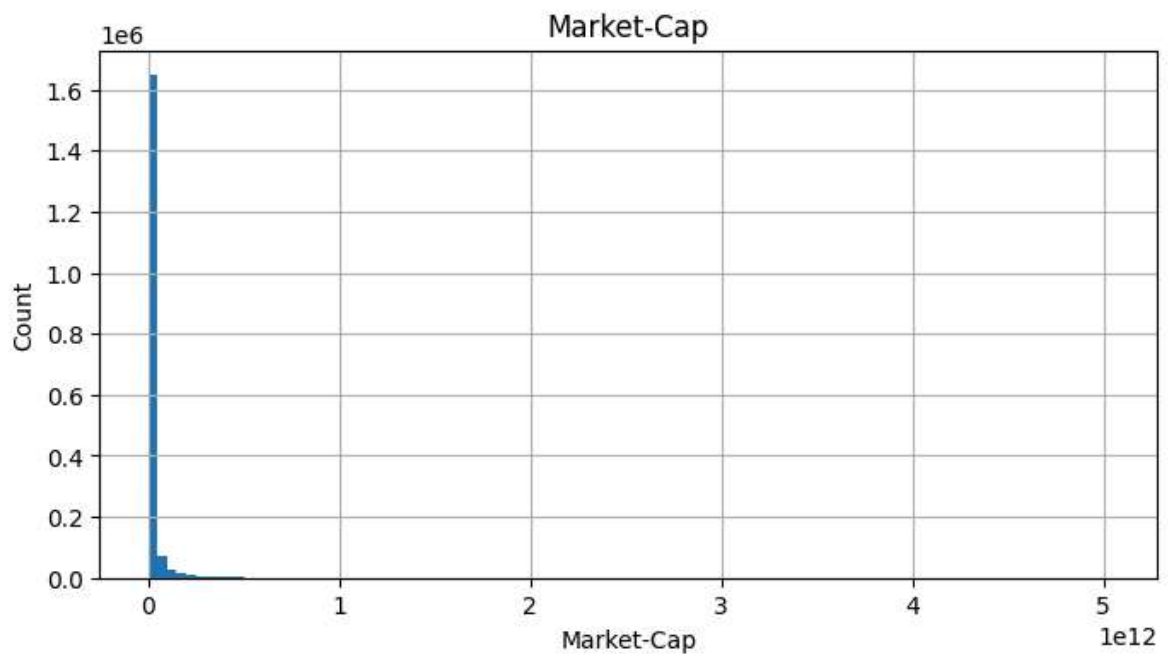
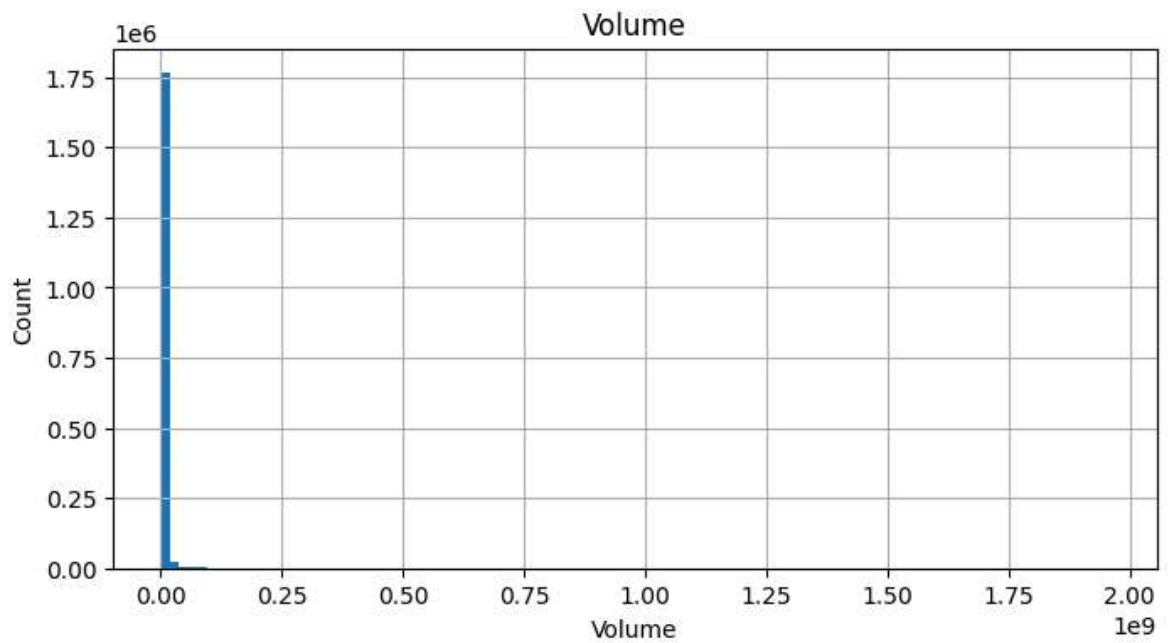
    df_good[col].hist(
        bins=100
    )

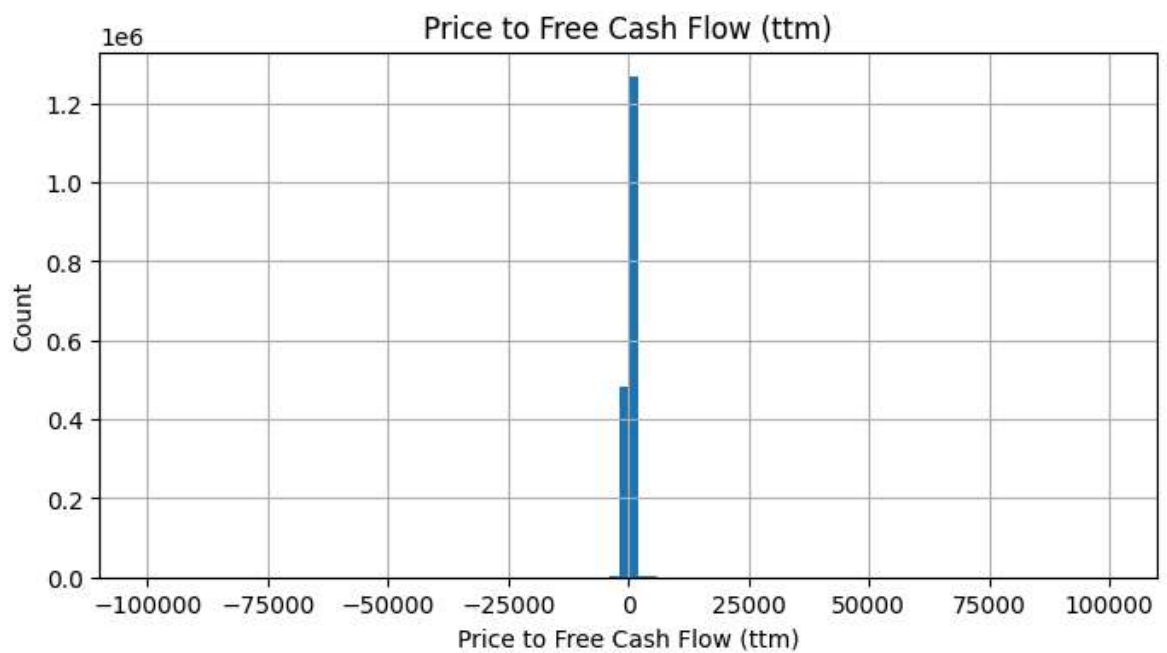
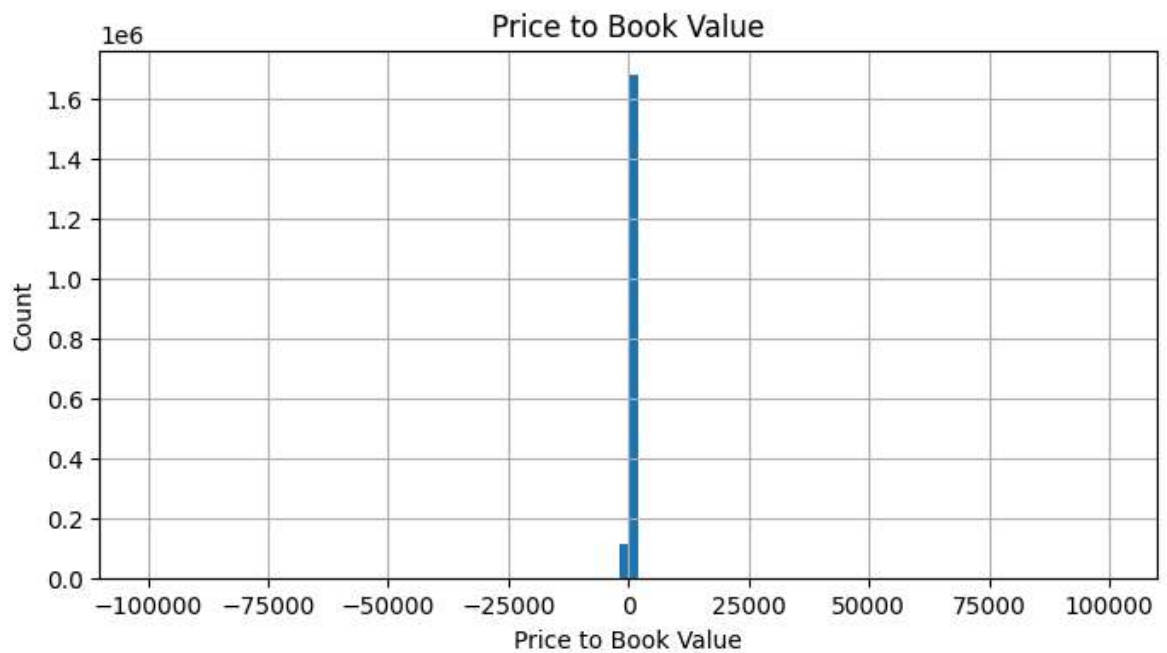
    plt.title(col)
    plt.xlabel(col)
    plt.ylabel("Count")

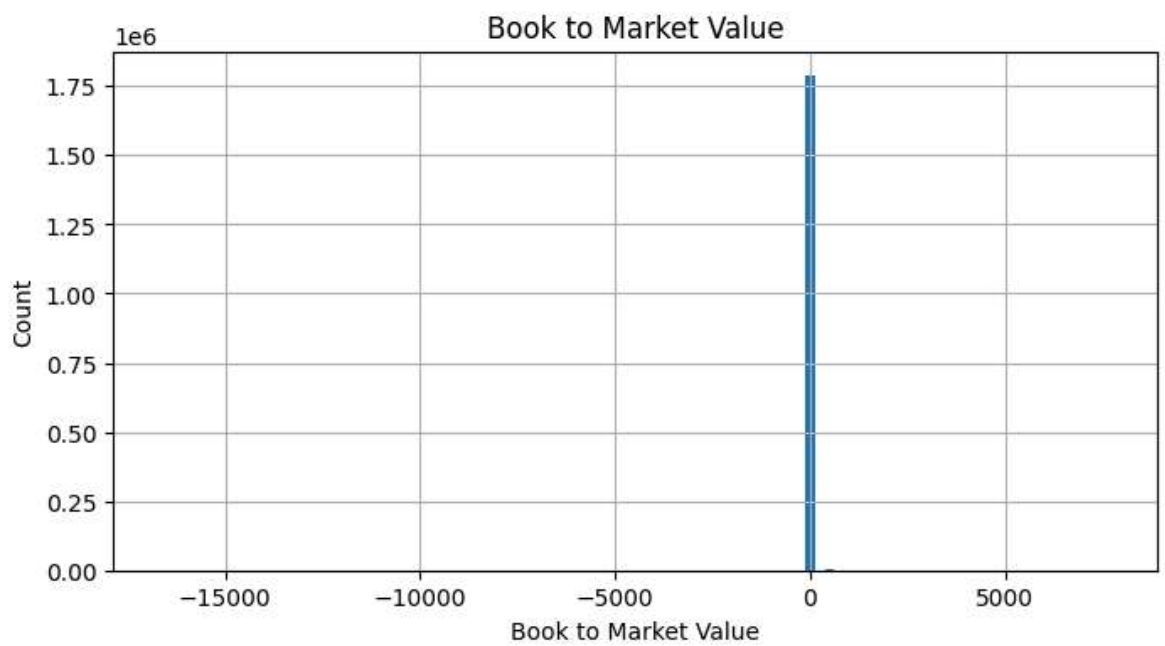
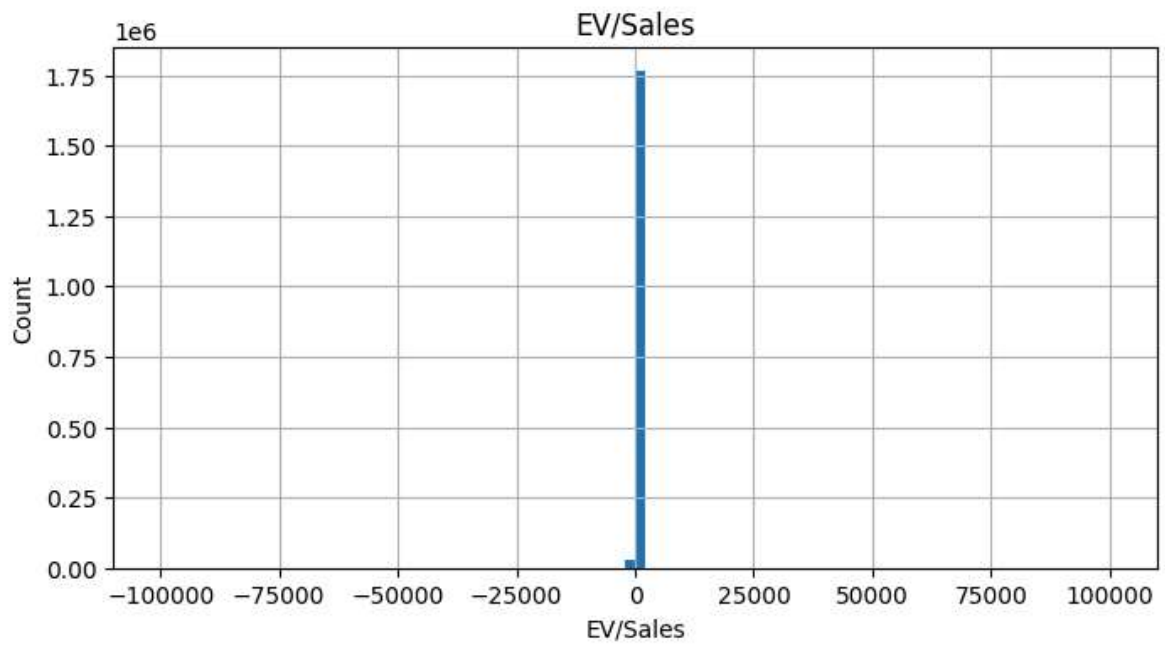
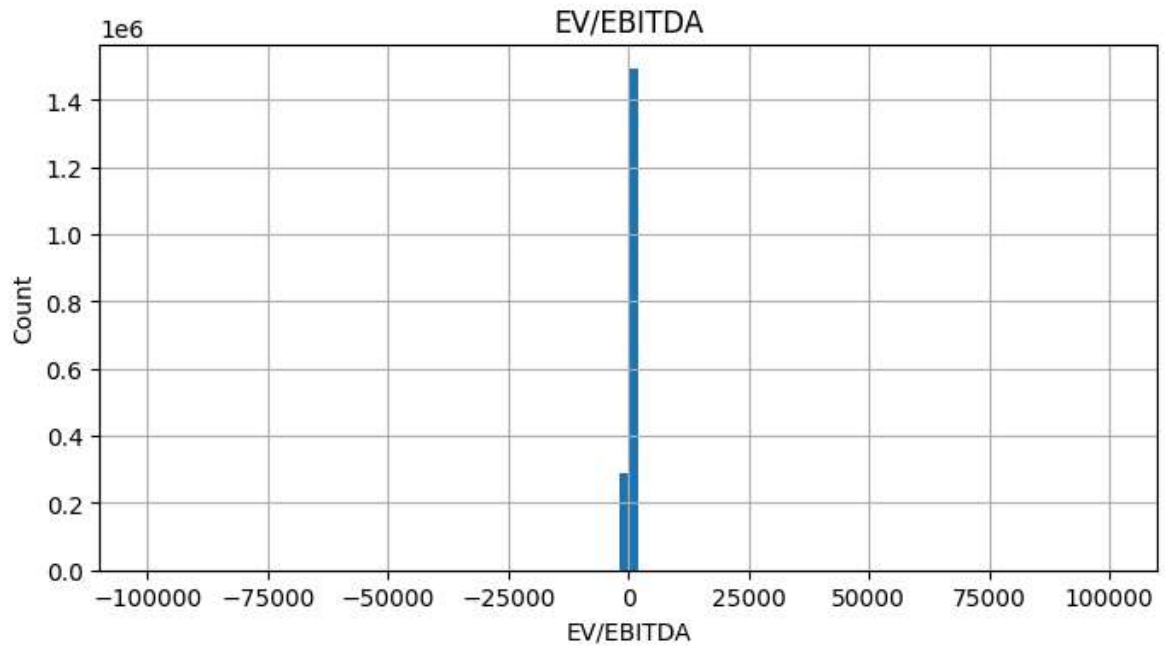
    plt.show()

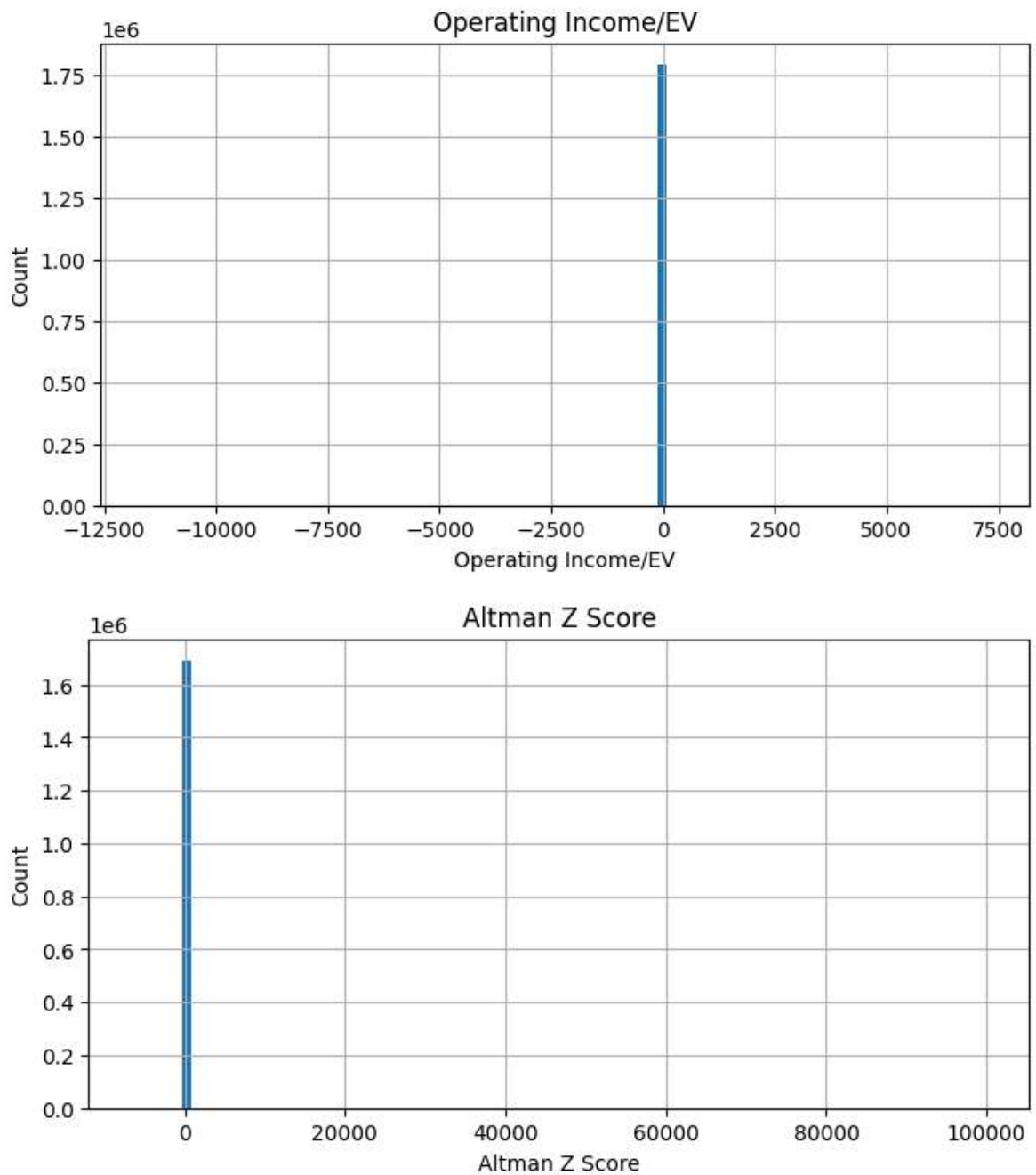
```











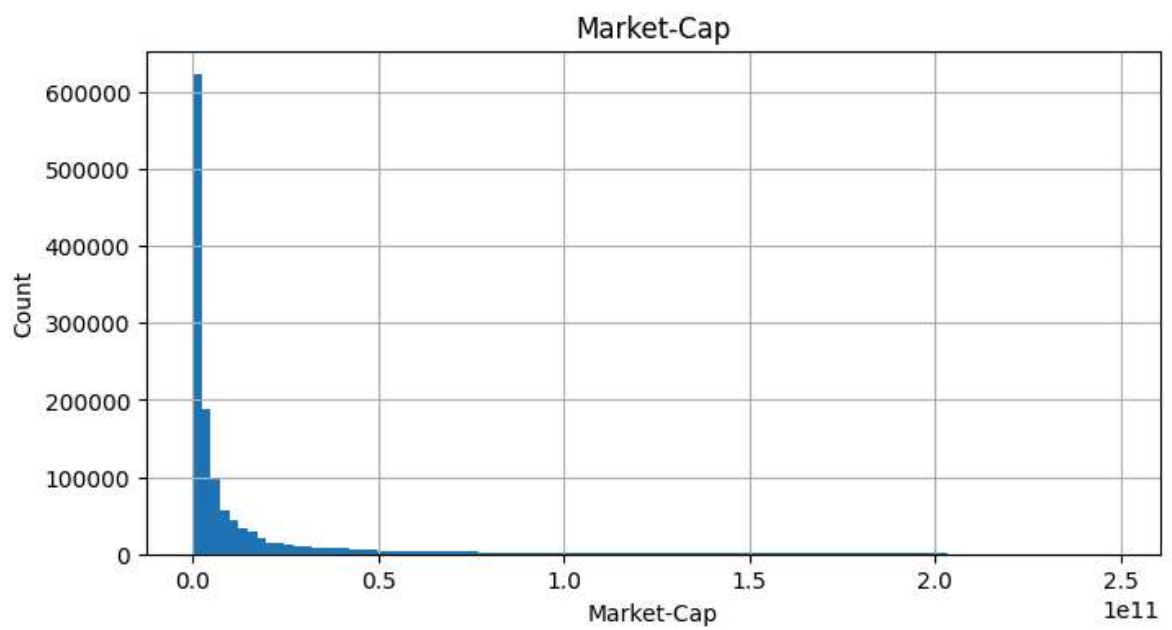
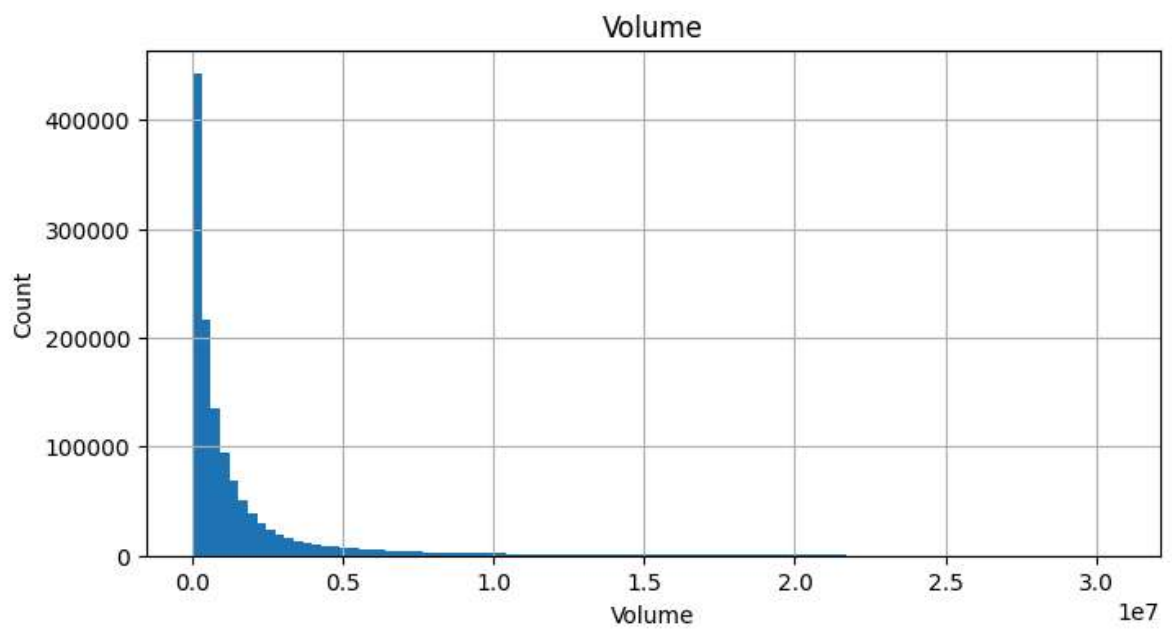
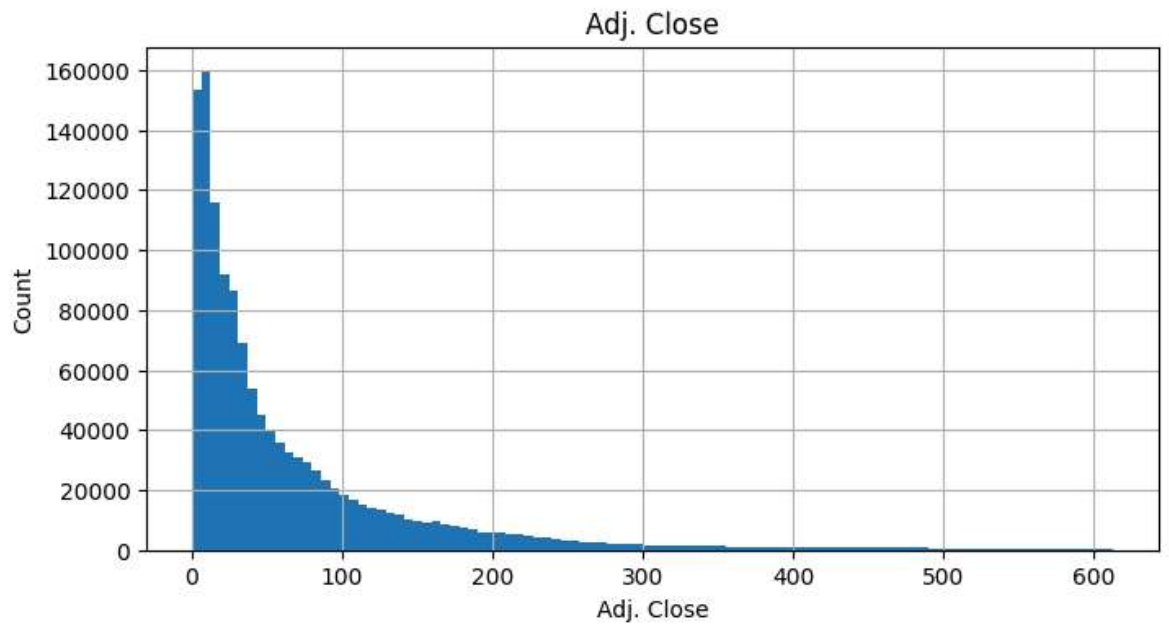
```
In [24]: for col in df_trim.columns:

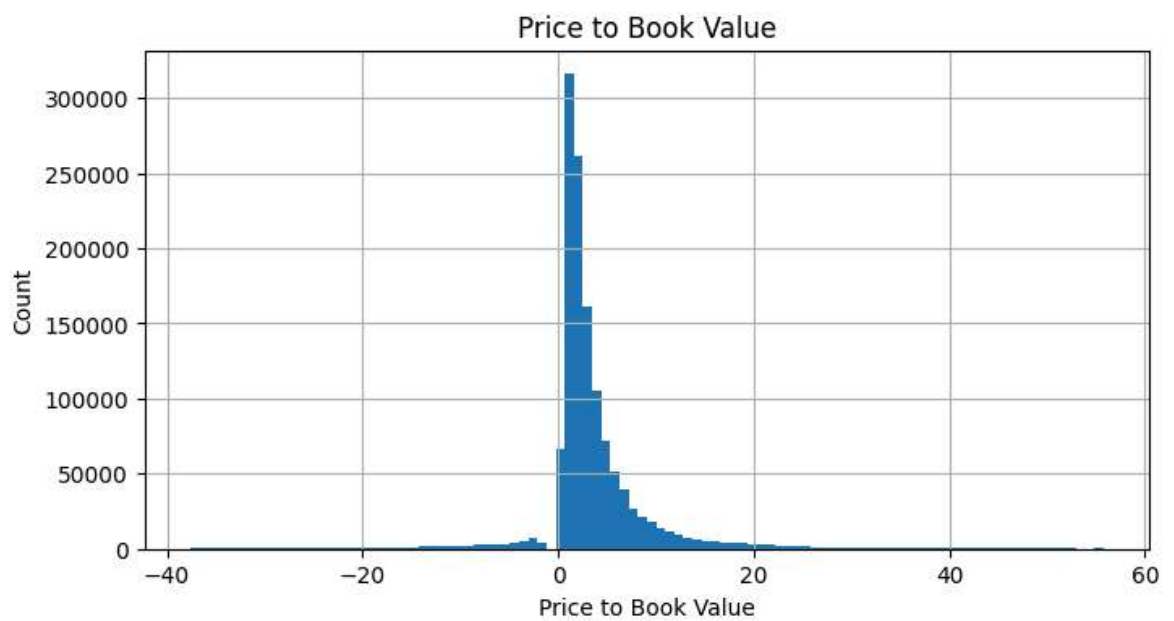
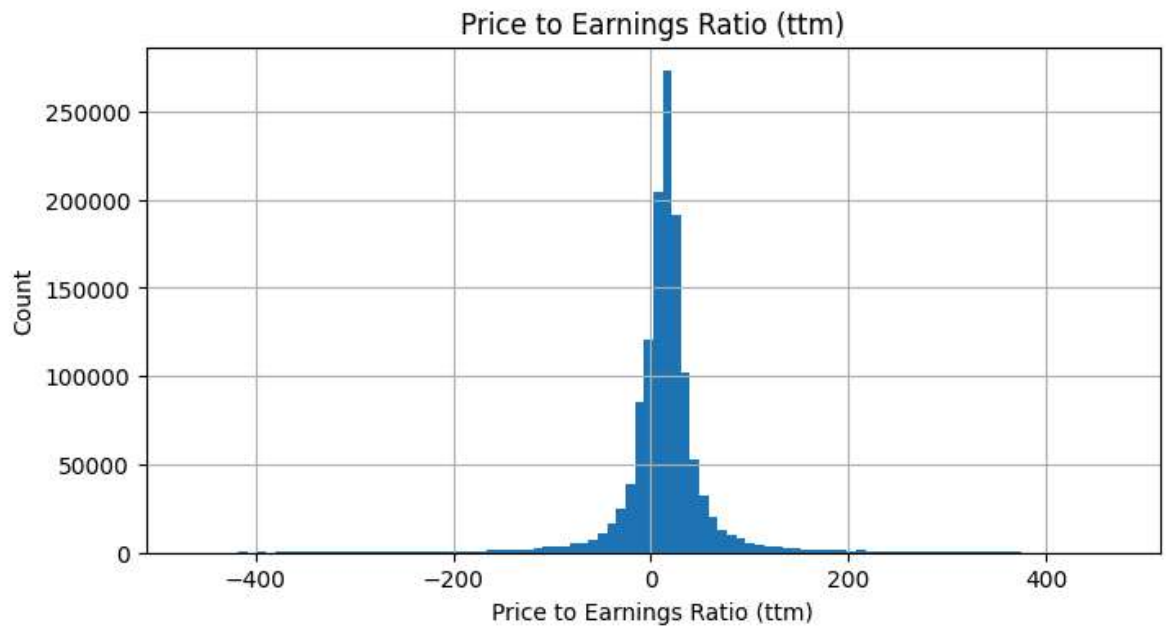
    plt.figure(figsize=(8, 4))

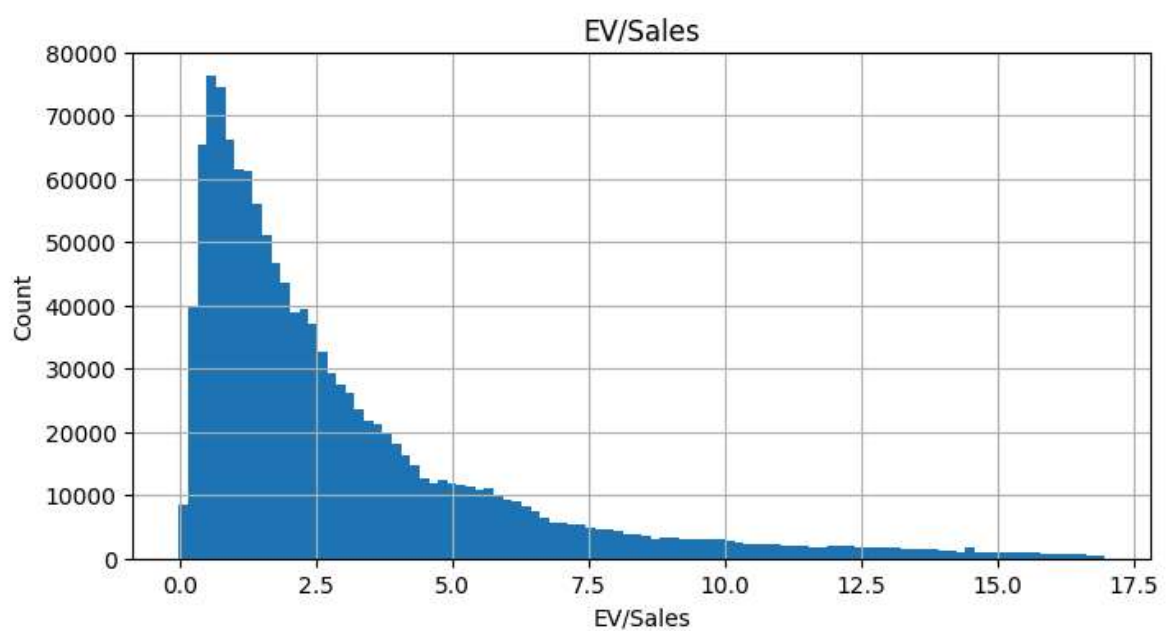
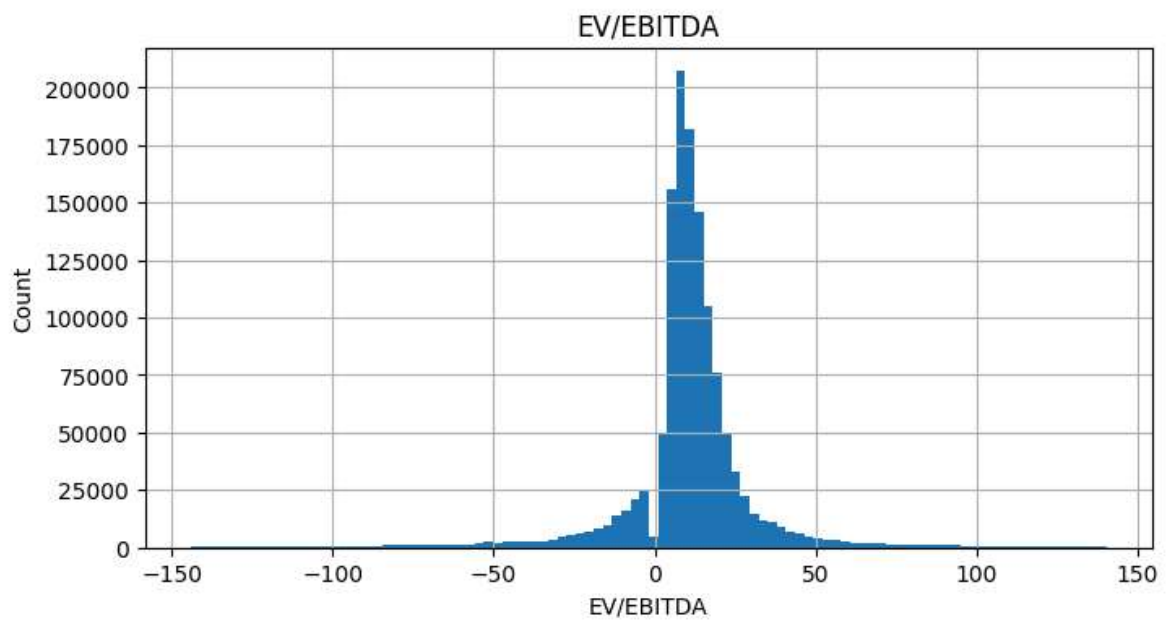
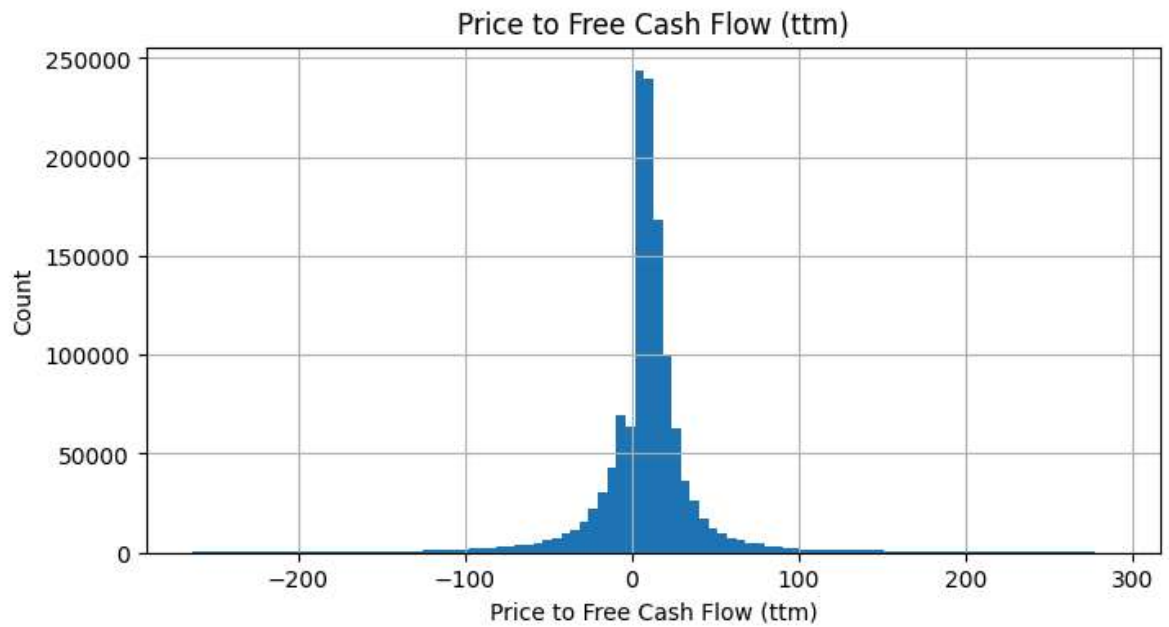
    df_trim[col].hist(
        bins=100
    )

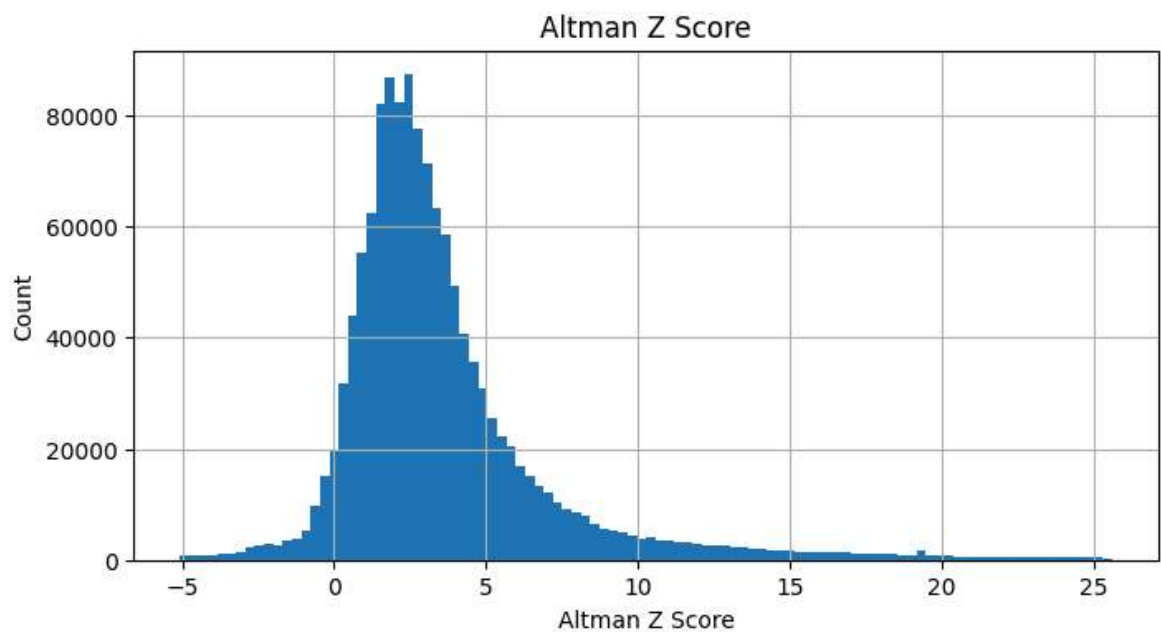
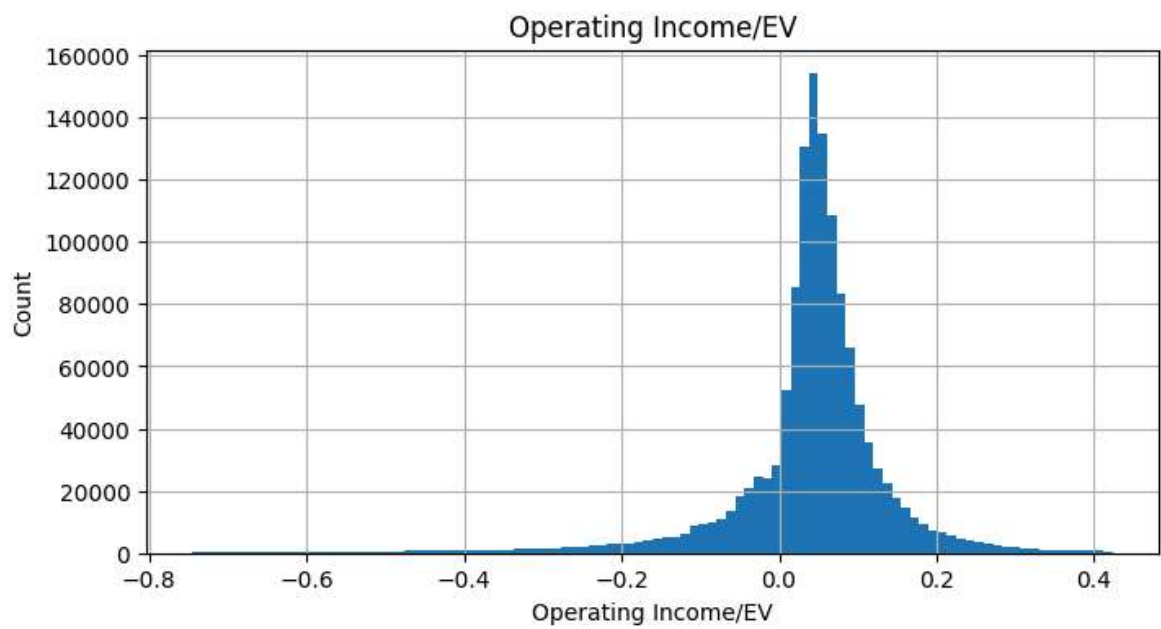
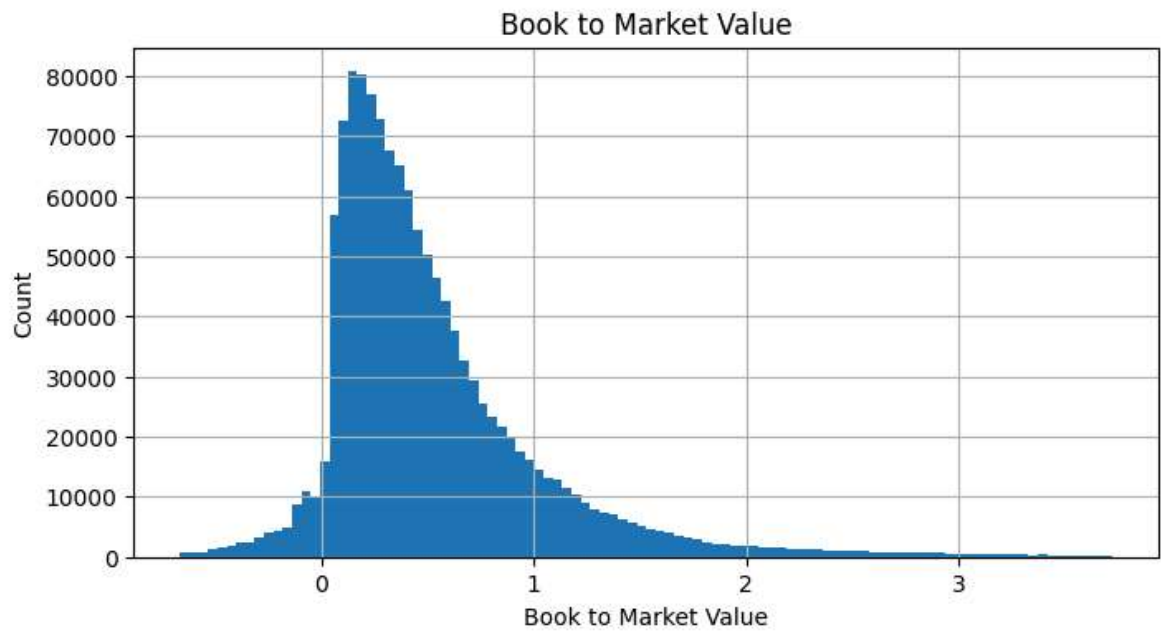
    plt.title(col)
    plt.xlabel(col)
    plt.ylabel("Count")

    plt.show()
```







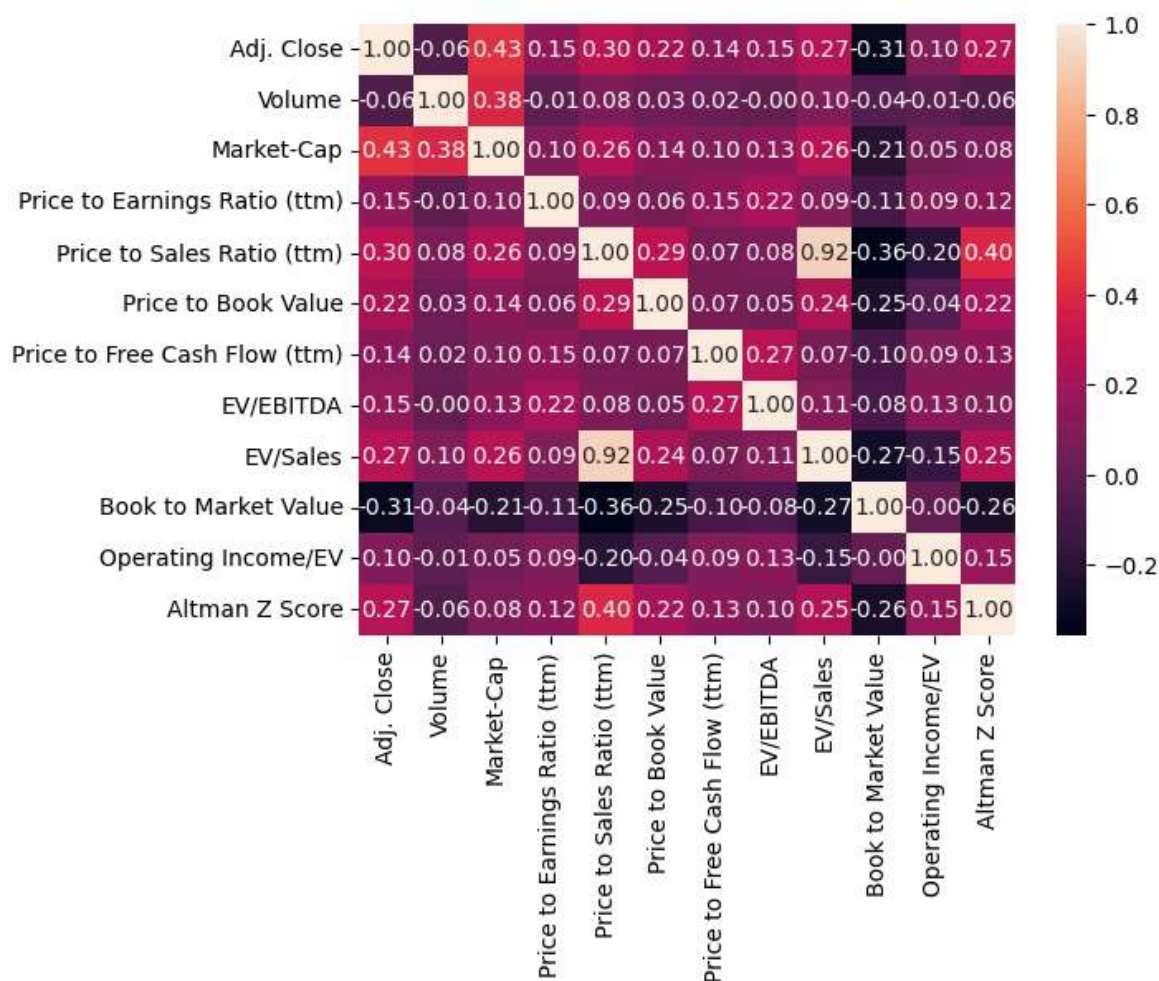


Above you can see difference in the graphs distributions from the dataframe before and after the cleaning and selection. There are a couple of valuable observations, of weird behaviour.

1. We can see that the range of values dramatically decreases
2. We can see a lot of features are heavily skewed which is normal and inline with what we expected.
3. Ideally we would like to see even curves in the graphs indicating a spreaded distribution. However in some features graphs we still see some unexplainable fluctuations that require further investigation. Like drops around the 0 value.
4. This is also strange since for some graphs we wouldn't expect 0 values but they still contain them and some of them quite a lot.

Feature Selection

```
In [25]: correlations = df_trim.corr()
plot = seaborn.heatmap(correlations, cbar=True, annot=True, fmt=".2f")
```



The final correlation heatmap shows noticeably stronger relationships between the financial ratios and the target variable (Adjusted Close) compared to the baseline

dataset. For example, the correlation between Adjusted Close and Price to Book Value increased from 0.22 to 0.44, while correlations with Price to Sales Ratio, EV/EBITDA, Market-Cap, and Altman Z Score also increased substantially.

At the same time, the relationships between the financial ratios themselves remained largely unchanged. For example, the strong relationship between Price to Sales Ratio and EV/Sales remained at 0.90, while the inverse relationship between Price to Book Value and Book to Market Value remained around -0.60.

This suggests that the cleaning process did not distort the underlying financial structure of the dataset. Instead, removing placeholder values and extreme outliers reduced noise, allowing the underlying relationships between the financial indicators and stock prices to become more visible.

```
In [26]: corr = df_trim.corr(numeric_only=True)["Adj. Close"]

print(corr.abs().sort_values(ascending=False).head(10))
```

Adj. Close	1.000000
Market-Cap	0.432477
Book to Market Value	0.312977
Price to Sales Ratio (ttm)	0.295118
EV/Sales	0.273738
Altman Z Score	0.273470
Price to Book Value	0.223199
EV/EBITDA	0.154956
Price to Earnings Ratio (ttm)	0.147125
Price to Free Cash Flow (ttm)	0.135335

Name: Adj. Close, dtype: float64

Above we see the top 10 correlations with our target variable. Since this is the baseline model we're gonna experiment and train 2 models on 2 feature sets. To try and see the differences. One of the feature sets is gonna include all features. The other feature set is gonna contain only features with a correlation above >0.25.

Also this confirms our hypothesis about improving the dataset to a usable dataset. So we will further use df_trim in the baseline model as our dataset.

Creating second dataset correlations above 0.25

```
In [27]: df_corr = df_trim.copy()

df_corr = df_corr[
    [
        "Market-Cap",
        "Book to Market Value",
        "Price to Sales Ratio (ttm)",
        "EV/Sales",
        "Altman Z Score",
        "Adj. Close"
    ]
]
```

```
]
]
```

Seperating target variable from dataset

```
In [28]: X_trim = df_trim.drop(columns=["Adj. Close"])
         y_trim = df_trim["Adj. Close"]
```

```
In [29]: X_corr = df_corr.drop(columns=["Adj. Close"])
         y_corr = df_corr["Adj. Close"]
```

Splitting train/validate/test

```
In [30]: from sklearn.model_selection import train_test_split

         X_trim_train, X_trim_test, y_trim_train, y_trim_test = train_test_split(X_trim, y
```

```
In [31]: X_corr_train, X_corr_test, y_corr_train, y_corr_test = train_test_split(X_corr, y
```

The dataset is divided into training, validation, and test sets. The training set is used for learning, the validation set monitors performance during training, and the test set evaluates final model performance on unseen data.

This separation helps ensure fair and reliable evaluation.

Model trainen

Model trainen op 2 verschillende feature sets een keer alle features en een keer alle features arbitrair boven 0.25

```
In [32]: from sklearn.linear_model import LinearRegression

         model = LinearRegression()
         result = model.fit(X_trim_train, y_trim_train)
         score = model.score(X_trim_test, y_trim_test)
         print("R²:", score)
```

R²: 0.2668905768154769

```
In [33]: model_corr = LinearRegression()
         result = model_corr.fit(X_corr_train, y_corr_train)
         score = model_corr.score(X_corr_test, y_corr_test)
         print("R²:", score)
```

R²: 0.18864499799485412

Hier hebben we het model getraind en zien we dat de feature set van alle features een stuk beter meer dan 25% is in voorspellen. Dan de correlatie set. Aangezien dit nog de baseline model is gaan we verder uitzoeken of we deze bevinding verder kunnen ondersteunen dat de set met alle features beter is als de correlatie set.

Evaluation

```
In [34]: # Predictions - all features / trim dataset
predictions_trim = model.predict(X_trim_test)

prediction_overview_trim = pd.DataFrame()
prediction_overview_trim["truth"] = y_trim_test
prediction_overview_trim["prediction"] = predictions_trim
prediction_overview_trim["error"] = prediction_overview_trim["truth"] - prediction_overview_trim["prediction"]
prediction_overview_trim["error"] = abs(prediction_overview_trim["error"]).astype(float)
prediction_overview_trim = prediction_overview_trim.reset_index(drop=True)

prediction_overview_trim
```

```
Out[34]:
```

	truth	prediction	error
0	63.68	77.788502	14
1	208.63	274.041600	65
2	3.66	17.501230	13
3	71.73	63.412251	8
4	14.92	43.679924	28
...	...	...	...
259333	65.60	61.209023	4
259334	38.59	58.834646	20
259335	23.28	57.828354	34
259336	17.84	55.033619	37
259337	17.86	77.657534	59

259338 rows × 3 columns

```
In [35]: # Predictions - correlation feature set
predictions_corr = model_corr.predict(X_corr_test)

prediction_overview_corr = pd.DataFrame()
prediction_overview_corr["truth"] = y_corr_test
prediction_overview_corr["prediction"] = predictions_corr
prediction_overview_corr["error"] = prediction_overview_corr["truth"] - prediction_overview_corr["prediction"]
prediction_overview_corr["error"] = abs(prediction_overview_corr["error"]).astype(float)
prediction_overview_corr = prediction_overview_corr.reset_index(drop=True)

prediction_overview_corr
```

Out[35]:

	truth	prediction	error
0	63.68	70.750795	7
1	208.63	242.620697	33
2	3.66	53.284560	49
3	71.73	58.902480	12
4	14.92	80.155719	65
...	...	...	...
259333	65.60	53.151431	12
259334	38.59	55.295622	16
259335	23.28	53.112177	29
259336	17.84	52.991342	35
259337	17.86	67.469755	49

259338 rows × 3 columns

Het lijkt erop dat het model underfit. We zien bij beide feature sets een toename in de errors bij lagere target variable waarden. Hier zal waarschijnlijk een duidelijk verklaarbare mogelijk op te lossen reden achter zitten die zeer interessant is verder uit te zoeken.

```
In [36]: # Error metrics - all features / trim dataset
me_trim = max_error(y_trim_test, predictions_trim)
me_trim = math.ceil(me_trim)
print("All Features Model")
print("Max Error:", me_trim)

mse_trim = mean_squared_error(y_trim_test, predictions_trim)
rmse_trim = math.sqrt(mse_trim)
rmse_trim = math.ceil(rmse_trim)
print("Root Mean Squared Error:", rmse_trim)
```

All Features Model

Max Error: 586

Root Mean Squared Error: 78

```
In [37]: # Error metrics - correlation feature set
me_corr = max_error(y_corr_test, predictions_corr)
me_corr = math.ceil(me_corr)
print("Correlation Feature Set Model")
print("Max Error:", me_corr)

mse_corr = mean_squared_error(y_corr_test, predictions_corr)
rmse_corr = math.sqrt(mse_corr)
rmse_corr = math.ceil(rmse_corr)
print("Root Mean Squared Error:", rmse_corr)
```

Correlation Feature Set Model

Max Error: 558

Root Mean Squared Error: 82

Als we de Max error en RMSE van beide feature sets vergelijken zien we geen duidelijke substantiele verschillen waaruit een duidelijke conclusie getrokken kan worden. De correlatie set doet het ietsjes beter. Maar zoals we net de grote verschillen zagen in de R2 scores is de verbetering hier lang niet genoeg om te verantwoorden verder met de correlatie set te gaan.

We kunnen dus wel stellen dat het lineare model wel degelijk extra waarde haalt uit ook de lage correlatie features en deze nog steeds waardevol blijven voor het model.

```
In [38]: plot_trim = pd.DataFrame({
    "Actual": y_trim_test,
    "Predicted": predictions_trim
})

plot_trim = plot_trim.sample(
    n=10000,
    random_state=42
)

plot_corr = pd.DataFrame({
    "Actual": y_corr_test,
    "Predicted": predictions_corr
})

plot_corr = plot_corr.sample(
    n=10000,
    random_state=42
)

fig, axes = plt.subplots(
    1, 2,
    figsize=(14,6)
)

axes[0].scatter(
    plot_trim["Actual"],
    plot_trim["Predicted"],
    alpha=0.3,
    s=5
)
axes[0].plot(
    [plot_trim["Actual"].min(), plot_trim["Actual"].max()],
    [plot_trim["Actual"].min(), plot_trim["Actual"].max()],
    "r--"
)
axes[0].set_title(
    "All Features (R2 = 0.267)"
)

axes[1].scatter(
    plot_corr["Actual"],
    plot_corr["Predicted"],
    alpha=0.3,
    s=5
)
axes[1].plot(
    [plot_corr["Actual"].min(), plot_corr["Actual"].max()],
```

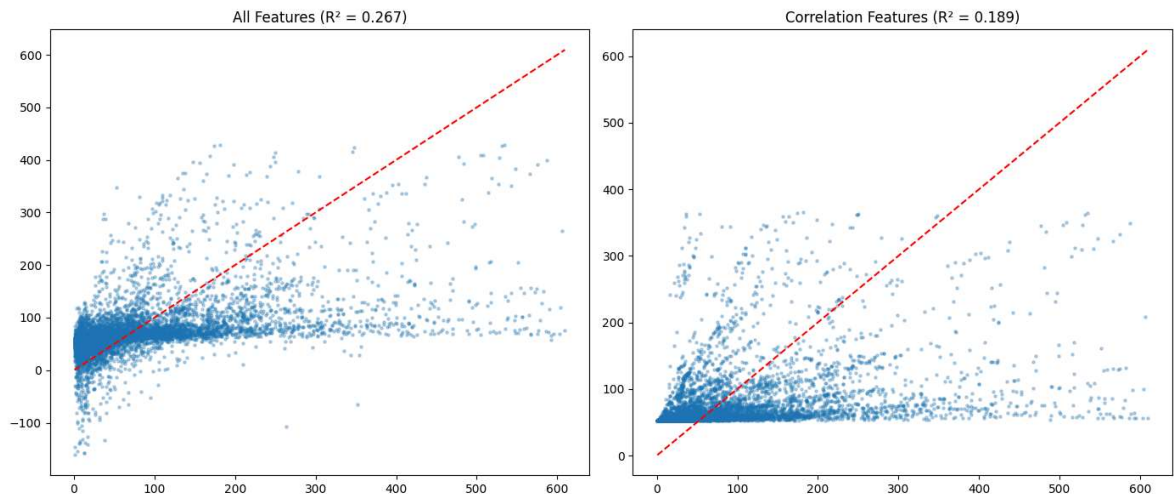


```

[plot_corr["Actual"].min(), plot_corr["Actual"].max()],
"r--"
)
axes[1].set_title(
    "Correlation Features ( $R^2 = 0.189$ )"
)

plt.tight_layout()
plt.show()

```



Het lijkt erop dat beide modellen een stuk beter zijn in het voorspellen van lagere waarde bedrijven als hogere waarde bedrijven.

```

In [39]: errors_trim = (
    plot_trim["Actual"] -
    plot_trim["Predicted"]
)

errors_corr = (
    plot_corr["Actual"] -
    plot_corr["Predicted"]
)

fig, axes = plt.subplots(
    1, 2,
    figsize=(14,6)
)

# All Features
axes[0].scatter(
    plot_trim["Predicted"],
    errors_trim,
    alpha=0.3,
    s=5
)

axes[0].axhline(
    0,
    color="red",
    linestyle="--"
)

axes[0].set_title(
    "All Features Residual Plot"
)

```

```

)

axes[0].set_xlabel(
    "Predicted Price"
)

axes[0].set_ylabel(
    "Residual Error"
)

# Correlation Features
axes[1].scatter(
    plot_corr["Predicted"],
    errors_corr,
    alpha=0.3,
    s=5
)

axes[1].axhline(
    0,
    color="red",
    linestyle="--"
)

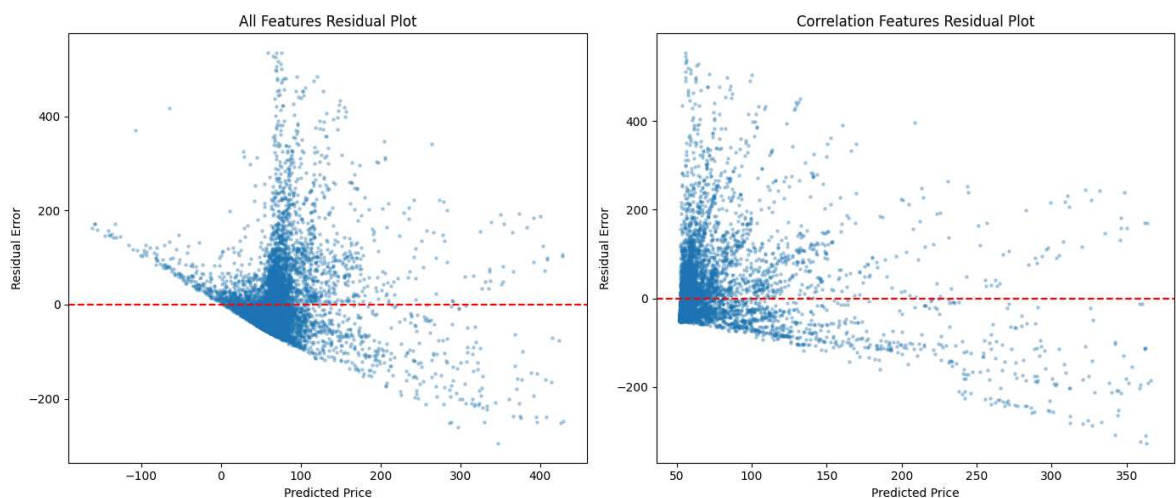
axes[1].set_title(
    "Correlation Features Residual Plot"
)

axes[1].set_xlabel(
    "Predicted Price"
)

axes[1].set_ylabel(
    "Residual Error"
)

plt.tight_layout()
plt.show()

```



Onze eerdere aanname van dat beide modellen beter zijn in voorspellen van lagere waarde prijzen bedrijven wordt hier ook bevestigd. In dat de distributie van de errors meer uitspreid naarmate de prijs toeneemt.

Overall Conclusion

Initial analysis showed almost no correlation between the financial metrics and stock price. Further investigation revealed significant data quality issues, including missing values, incomplete company records, placeholder values, and extreme outliers.

After filtering companies with sufficient data completeness and removing the upper and lower 1% of observations, much stronger and more economically meaningful relationships emerged. This confirmed that outliers were masking underlying patterns in the dataset.

A baseline Linear Regression model achieved an R^2 score of 0.267. An experiment using only the strongest correlated features resulted in a lower R^2 score of 0.189, indicating that features with weak individual correlations still contribute useful information when combined with other variables.

Overall, the notebook demonstrated the importance of data quality and outlier treatment and established a baseline model that can be improved in future iterations through more advanced feature selection, modelling techniques, and a future stock price target variable.